

**UNIVERSIDAD AUTÓNOMA
GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA Y CIENCIAS
DE LA COMPUTACIÓN Y TELECOMUNICACIONES**



Primer Examen Parcial

**SOFTWARE WEB Y MÓVIL: PLATAFORMA INTELIGENTE DE
ATENCIÓN DE EMERGENCIAS VEHICULARES Y AUXILIO
MECÁNICO**

Nro. Grupo: 18

Materia: Sistemas de Información II - SB

Docente: Ing. Rolando Martínez Canedo

Integrantes:

- Cervantes Arancibia Roberto Carlos 222007893
- Jiménez Peña Marcelo 222008751

Fecha de presentación: miércoles 29 de abril de 2026
Santa Cruz – Bolivia

CAPÍTULO 1: PERFIL DEL PROYECTO.....	5
1.1 Introducción.....	5
1.2 Antecedentes.....	6
1.3 Descripción del problema.....	7
1.4 Objetivos.....	9
1.4.1 Objetivo general.....	9
1.4.2 Objetivos específicos.....	9
1.5 Justificación.....	10
1.6 Alcance.....	11
1.6.1 Aplicación móvil (para clientes).....	11
1.6.2 Aplicación web (para talleres).....	12
1.6.3 Servicios de inteligencia artificial (integrados en el backend).....	13
1.7 Sistema de información basado en computadoras.....	14
1.7.1 Hardware.....	14
1.7.2 Software.....	15
1.7.3 Datos.....	17
1.7.4 Procesos.....	18
1.7.5 Gente.....	18
1.7.6 Documentación.....	19
1.8 Tecnología.....	19
1.9 Costos estimados.....	20
1.10 Beneficios.....	21
CAPÍTULO 2: FUNDAMENTACIÓN TEÓRICA.....	22
2.1 Concepto de auxilio mecánico vehicular.....	22
2.2 Importancia de la digitalización del servicio.....	22
2.3 Componentes clave del servicio.....	23
2.4 Flujo operativo general del servicio.....	24
2.5 Retos funcionales del dominio.....	24
2.6 Referentes funcionales y estado del arte.....	24
2.6.1 Plataformas de intermediación bajo demanda.....	24
2.6.2 Servicios tradicionales de asistencia vial.....	25
2.6.3 Directorios digitales y marketplaces de talleres.....	25
2.6.4 Posicionamiento de la propuesta.....	25
2.7 Uso de la inteligencia artificial en la plataforma.....	26
2.7.1 Rol de la inteligencia artificial dentro del sistema.....	26
2.7.2 Procesamiento de audio.....	26
2.7.3 Clasificación preliminar de incidentes.....	26
2.7.4 Análisis básico de imágenes.....	27
2.7.5 Generación de resumen del incidente.....	27
2.7.6 Sistema de priorización y asignación.....	27
2.7.7 Beneficios del enfoque inteligente.....	28
2.8 Herramientas tecnológicas.....	28
2.8.1 FastAPI.....	28

2.8.2 Angular.....	28
2.8.3 Flutter.....	28
2.8.4 PostgreSQL.....	28
2.8.5 AWS.....	29
2.8.6 GitHub y control de versiones.....	29
2.8.7 Herramientas complementarias.....	29
2.9 Proceso Unificado de Desarrollo de Software (PUDS).....	29
2.9.1 Concepto general.....	29
2.9.2 Fases del PUDS.....	29
2.9.3 Pertinencia del PUDS para el proyecto.....	30
2.10 UML.....	30
2.10.1 Definición.....	30
2.10.2 Propósitos de UML en el proyecto.....	30
2.10.3 Diagramas recomendados para el proyecto.....	31
2.10.4 Importancia académica y técnica.....	31
CAPÍTULO 3: PROCESO DE DESARROLLO DEL SOFTWARE.....	31
3.1 FT: CAPTURA DE REQUISITOS.....	31
3.1.1 Identificar actores.....	31
3.1.2 Identificar casos de uso.....	32
3.1.3 Priorización de casos de uso.....	33
3.1.4 Especificar casos de uso.....	34
Ciclo #1.....	35
CU01 Registrar usuario.....	35
CU02 Iniciar sesión.....	36
CU03 Registrar vehículo.....	37
CU04 Reportar emergencia.....	38
CU05 Adjuntar evidencias.....	39
CU08 Gestionar información del taller.....	40
CU09 Gestionar técnicos.....	41
CU11 Visualizar solicitudes disponibles.....	42
CU12 Aceptar o rechazar solicitud.....	43
CU13 Asignar técnico.....	44
CU14 Gestionar atención del servicio.....	44
Ciclo #2.....	45
CU06 Consultar estado de solicitud.....	45
CU07 Recibir notificaciones push del servicio.....	47
CU10 Gestionar disponibilidad.....	48
CU15 Generar resumen del incidente.....	49
CU16 Analizar evidencias del incidente.....	50
CU17 Clasificar y priorizar incidente.....	51
Ciclo #3.....	52
CU18 Seleccionar talleres candidatos.....	52
CU19 Registrar pago.....	53
CU20 Calificar servicio.....	54

CU21 Consultar historial de atenciones.....	55
CU22 Gestionar métricas y reportes.....	56
3.1.5 Estructurar Modelo de Casos de Uso.....	57
Ciclo #1.....	57
Ciclo #2.....	58
Ciclo #3.....	59
3.2 FT: ANÁLISIS.....	59
3.2.1 Identificar paquetes.....	59
3.2.2 Relacionar paquetes y casos de uso.....	61
3.2.3 Diagramas de Comunicación/Colaboración.....	65
3.2.4 Análisis de Paquetes.....	68
3.3 FT: DISEÑO.....	69
3.3.1 Diseño de la arquitectura.....	69
3.3.1.1 Diseño físico (Diagrama de Despliegue).....	69
3.3.1.2 Diseño lógico (Diagrama Organizado en Capas).....	69
3.3.2 Diseño de Datos.....	70
3.3.2.1 Diseño de Datos Lógicos.....	70
3.3.2.1.1 Diagrama de Clases.....	70
3.3.2.1.2 Mapeo.....	70
3.3.2.1.3 Normalización.....	70
3.3.3 Diagramas de Secuencia.....	70
3.4 FT: IMPLEMENTACIÓN.....	70
3.4.1 Elección de plataforma de desarrollo de software.....	70
3.4.2 Implementación de la arquitectura del sistema.....	71
3.4.3 Endpoints sugeridos.....	72
3.4.4 Organización inicial de repositorios.....	72
Se recomienda que cada repositorio posea su propio README, instrucciones de instalación, variables de entorno documentadas y convenciones mínimas de ramas para desarrollo colaborativo.....	73
3.4.5 Estructura base sugerida por repositorio.....	73
3.5 Pruebas.....	74
3.5.1 Estrategia de pruebas.....	74
3.5.2 Casos de prueba sugeridos.....	74
3.5.3 Criterios de aceptación del MVP.....	75
CAPÍTULO 4: MANUAL DE USUARIO.....	76
4.1 Introducción.....	76
4.2 Acceso a la aplicación.....	76
4.2.1 Versión móvil – clientes.....	76
4.2.2 Versión web – talleres.....	76
4.3 Uso de funcionalidades.....	77
4.3.1 Registro de vehículo.....	77
4.3.2 Registro de emergencia.....	77
4.3.3 Consulta del estado del servicio.....	77
4.3.4 Aceptación de solicitud por el taller.....	77

4.3.5 Asignación del técnico.....	77
4.3.6 Actualización de estado.....	78
4.3.7 Registro de pago y calificación.....	78
4.4 Notificaciones.....	78
4.5 Recomendaciones de uso.....	78
CONCLUSIONES.....	78
RECOMENDACIONES.....	79
BIBLIOGRAFÍA.....	80
URL Y QR.....	80
ANEXOS.....	81

CAPÍTULO 1: PERFIL DEL PROYECTO

1.1 Introducción

En la actualidad, la movilidad urbana e interprovincial se ha convertido en un componente esencial de la vida cotidiana de las personas, tanto para actividades laborales como personales. Sin embargo, junto con el incremento del uso de vehículos también han aumentado las situaciones imprevistas en carretera y en ciudad, tales como fallas mecánicas, pinchazos de llanta, problemas de batería, sobrecalentamiento del motor, fallas eléctricas, accidentes leves, pérdida de llaves o necesidad de remolque. Frente a estas emergencias, la obtención de auxilio continúa dependiendo en muchos casos de llamadas telefónicas, contactos informales, búsquedas manuales en redes sociales o recomendaciones poco verificables, lo que ocasiona tiempos de respuesta inciertos, diagnósticos poco claros y una atención desorganizada.

Por otra parte, los talleres mecánicos y proveedores de auxilio vial generalmente no cuentan con una plataforma centralizada que les permita recibir solicitudes estructuradas, filtrar incidentes según su especialidad, analizar la ubicación del cliente, evaluar la disponibilidad de técnicos y aceptar atenciones en tiempo real. Esta carencia tecnológica provoca una asignación deficiente de recursos, pérdida de oportunidades de servicio y una experiencia insatisfactoria para el usuario final.

En este contexto surge la propuesta de desarrollar una plataforma inteligente de atención de emergencias vehiculares y auxilio mecánico, integrada por una aplicación móvil para clientes, una aplicación web para talleres y un backend centralizado capaz de procesar información multimodal. El sistema permitirá que el cliente registre una emergencia mediante texto, audio, fotografías y geolocalización, para que posteriormente el sistema clasifique preliminarmente el incidente, calcule prioridad, notifique a los talleres más adecuados y facilite la asignación del servicio. De esta forma, se busca transformar un proceso totalmente manual y reactivo en un servicio digital, trazable, inteligente y orientado a la atención oportuna.

La propuesta responde además a un escenario actual donde la digitalización, la geolocalización en tiempo real y la inteligencia artificial ya no constituyen elementos accesorios, sino capacidades estratégicas para mejorar la eficiencia operativa, reducir los tiempos de respuesta y optimizar la experiencia del usuario. El proyecto integra tecnologías modernas como FastAPI, Angular, Flutter, PostgreSQL y AWS, además de servicios complementarios para almacenamiento, notificaciones y procesamiento de datos, con el fin de ofrecer una solución robusta, escalable y alineada con las exigencias de una plataforma moderna de asistencia vehicular.

1.2 Antecedentes

El sector de servicios automotrices y auxilio vehicular en Bolivia todavía presenta un importante nivel de informalidad en la manera en que los usuarios encuentran asistencia cuando se enfrentan a una emergencia. En muchos casos, el conductor depende de recomendaciones personales, búsquedas en redes sociales, llamadas telefónicas directas o contactos previos con talleres conocidos. Este enfoque genera varios inconvenientes: falta de trazabilidad, poca claridad sobre la especialidad real del taller, demoras en la atención y ausencia de mecanismos para verificar reputación, tiempos estimados, costos o disponibilidad del personal.

A nivel operativo, muchos talleres organizan su trabajo de manera manual o semimanual. La recepción de solicitudes, la asignación de técnicos, la gestión de disponibilidad y el seguimiento del servicio suelen depender de llamadas, mensajes o coordinación interna no estructurada. Como consecuencia, los talleres tienen dificultades para priorizar casos urgentes, distribuir correctamente a sus técnicos, registrar historiales de atención y aprovechar mejor su capacidad operativa.

En los últimos años han surgido plataformas digitales de intermediación en otros sectores, como transporte, delivery y comercio electrónico, que han demostrado la importancia de conectar usuarios con

proveedores mediante aplicaciones móviles y sistemas centralizados. Este mismo principio puede ser aplicado al ámbito del auxilio mecánico, incorporando además componentes de inteligencia artificial para asistir en el diagnóstico preliminar del problema a partir de imágenes, audio, texto y ubicación. De esta manera, el sistema no solo intermedia entre cliente y taller, sino que mejora la calidad de la información con la que se toma la decisión de atención.

Desde una perspectiva tecnológica, la evolución de los servicios cloud, las APIs de geolocalización, las notificaciones push y los modelos de clasificación de datos multimodales permite construir una solución de alto valor académico y práctico. Por tanto, este proyecto se ubica en la convergencia entre sistemas de información, aplicaciones distribuidas, procesamiento inteligente de datos y servicios en tiempo real, constituyéndose en una propuesta pertinente para la materia Sistemas de Información II.

1.3 Descripción del problema

La problemática central radica en la ausencia de una plataforma digital inteligente que permita gestionar de manera eficiente las emergencias vehiculares y conecte, en tiempo real, a los conductores con los talleres o proveedores de auxilio más adecuados según la naturaleza del incidente, la ubicación y la disponibilidad del servicio.

Actualmente, cuando un conductor sufre una falla mecánica o incidente leve, enfrenta varias limitaciones. Primero, no existe un mecanismo estructurado para registrar el problema con suficiente contexto. Generalmente la información se comunica verbalmente y de forma incompleta, lo que dificulta identificar si se trata de una falla de batería, un pinchazo de llanta, una avería eléctrica, una necesidad de remolque o un incidente más complejo. Segundo, el usuario no cuenta con una forma confiable de saber qué taller o técnico está disponible, cuál tiene experiencia en el problema reportado o cuánto tiempo tardará en brindar atención.

Del lado de los talleres, el problema también es relevante. Sin una plataforma centralizada, los talleres no pueden recibir incidentes de forma organizada ni evaluarlos rápidamente según especialidad, urgencia, ubicación del cliente y disponibilidad de personal. Esto se traduce en asignaciones ineficientes, saturación de algunos talleres, desaprovechamiento de otros y tiempos de respuesta mayores a los necesarios. A ello se suma la ausencia de mecanismos de reputación, historial de atenciones, clasificación de incidentes y seguimiento del estado del servicio.

La situación se agrava cuando se considera que muchas emergencias requieren decisiones rápidas. Un incidente que inicialmente parece menor puede requerir remolque, herramientas específicas o un técnico especializado. Sin apoyo tecnológico, la clasificación del caso queda enteramente sujeta a la interpretación del usuario o del taller, incrementando el riesgo de diagnósticos imprecisos y atención inadecuada.

En síntesis, el problema se expresa en cinco dimensiones principales:

1. Dificultad para registrar emergencias de forma estructurada y completa.
2. Falta de mecanismos inteligentes para clasificar preliminarmente el incidente.
3. Ausencia de asignación eficiente de talleres y técnicos según contexto real.
4. Escasa trazabilidad del servicio, desde la solicitud hasta la finalización.
5. Experiencia deficiente para cliente y taller por procesos manuales y fragmentados.

Frente a ello, se plantea como solución una plataforma inteligente de auxilio mecánico que integre aplicación móvil, aplicación web, backend, geolocalización, notificaciones en tiempo real y módulos de inteligencia

artificial, con el propósito de mejorar la velocidad, precisión y calidad del servicio prestado.

1.4 Objetivos

1.4.1 Objetivo general

Desarrollar una plataforma inteligente de atención de emergencias vehiculares que permita conectar usuarios con talleres mecánicos mediante el análisis automatizado de incidentes usando datos multimodales —imagen, audio, texto y geolocalización—, optimizando el diagnóstico preliminar, la priorización del caso y la asignación del servicio.

1.4.2 Objetivos específicos

1. Diseñar una arquitectura basada en servicios que soporte procesamiento en tiempo real para el registro, clasificación y seguimiento de emergencias vehiculares.
2. Implementar una aplicación móvil para clientes que permita registrar vehículos, reportar incidentes, adjuntar evidencias y visualizar el estado del servicio.
3. Diseñar una aplicación web para talleres que permita gestionar solicitudes, disponibilidad, técnicos, historial de atenciones y estado operativo.
4. Integrar mecanismos de geolocalización para registrar la ubicación del incidente y facilitar la selección del taller más adecuado.
5. Incorporar módulos de apoyo inteligente para transcripción de audio, clasificación preliminar de incidentes, análisis básico de imágenes y generación de resumen estructurado.
6. Diseñar un sistema de priorización que considere tipo de problema, nivel de urgencia, contexto y disponibilidad de recursos.
7. Implementar un mecanismo de asignación inteligente de talleres y técnicos basado en cercanía, especialidad, capacidad operativa y prioridad del caso.

8. Gestionar notificaciones push y actualizaciones en tiempo real para clientes y talleres durante todo el ciclo de atención.
9. Mantener trazabilidad completa de cada incidente, desde su registro hasta el cierre del servicio y la calificación del cliente.
10. Desplegar el sistema utilizando tecnologías modernas y servicios cloud que garanticen disponibilidad, escalabilidad y seguridad.

1.5 Justificación

La implementación de esta plataforma se justifica desde una perspectiva social, operativa, tecnológica y académica.

Justificación social: los conductores requieren soluciones rápidas, confiables y seguras cuando enfrentan una emergencia vehicular. Una plataforma digital que reduzca tiempos de incertidumbre, facilite la obtención de asistencia y mejore la coordinación con talleres contribuye directamente a la seguridad y tranquilidad del usuario.

Justificación operativa: el sistema permitirá a los talleres organizar mejor la recepción de incidentes, asignar recursos disponibles de manera eficiente, priorizar casos y registrar la trazabilidad completa del servicio. Esto mejora la productividad del taller, reduce tiempos improductivos y profesionaliza la atención.

Justificación tecnológica: el proyecto integra aplicaciones web y móviles, geolocalización, APIs REST, almacenamiento en la nube, notificaciones push y módulos de inteligencia artificial. Esta combinación hace posible una solución moderna, escalable y alineada con tendencias actuales de desarrollo de software orientado a servicios.

Justificación académica: el proyecto resulta pertinente para Sistemas de Información II porque permite aplicar principios de análisis, diseño y desarrollo de sistemas de información, así como modelado UML, PUDS, arquitectura por capas o servicios, base de datos relacional y despliegue en

la nube. Además, integra un componente innovador al incorporar IA dentro del flujo principal del sistema y no como un módulo aislado.

1.6 Alcance

El sistema estará compuesto por tres grandes áreas funcionales: aplicación móvil para clientes, aplicación web para talleres y servicios inteligentes integrados en el backend. Su alcance se describe a continuación.

1.6.1 Aplicación móvil (para clientes)

1. Módulo de autenticación y perfil

Funcionalidades clave:

- Registro de usuario.
- Inicio y cierre de sesión.
- Gestión de perfil personal.
- Recuperación de contraseña.

2. Módulo de vehículos

Funcionalidades clave:

- Registro de uno o varios vehículos.
- Edición de datos del vehículo.
- Asociación del vehículo a solicitudes de emergencia.

3. Módulo de registro de emergencia

Funcionalidades clave:

- Envío de ubicación en tiempo real.
- Adjuntar fotos del vehículo o del daño visible.
- Enviar audio describiendo el problema.
- Ingresar texto adicional opcional.
- Seleccionar el vehículo afectado.

4. Módulo de seguimiento del servicio

Funcionalidades clave:

- Visualizar el estado de la solicitud.
- Ver taller asignado.
- Ver técnico asignado, si corresponde.
- Consultar el tiempo estimado de llegada.
- Recibir actualizaciones en tiempo real.

5. Módulo de pagos y calificación

Funcionalidades clave:

- Registrar o efectuar el pago del servicio.
- Visualizar costo estimado y costo final.
- Seleccionar método de pago.
- Calificar la atención recibida.
- Consultar historial de emergencias atendidas.

1.6.2 Aplicación web (para talleres)

1. Módulo de gestión del taller

Funcionalidades clave:

- Gestión de la información del taller.
- Administración de datos generales.
- Configuración de especialidades y áreas de servicio.
- Gestión de reputación e historial.

2. Módulo de técnicos y disponibilidad

Funcionalidades clave:

- Gestión de técnicos.
- Asociación de especialidades a cada técnico.
- Gestión de disponibilidad.
- Consulta de carga operativa actual.

3. Módulo de solicitudes de asistencia

Funcionalidades clave:

- Visualización de solicitudes disponibles.
- Consulta estructurada del incidente.

- Aceptación o rechazo de solicitudes.
- Asignación de técnico.
- Actualización del estado del servicio.

4. Módulo de operación y seguimiento

Funcionalidades clave:

- Gestión de atenciones activas.
- Historial de servicios atendidos.
- Control de tiempos de atención.
- Visualización de resumen automático generado por IA.
- Consulta de clasificación preliminar y prioridad.

5. Módulo administrativo y financiero

Funcionalidades clave:

- Visualización de cobros realizados.
- Seguimiento de pagos realizados.
- Reportes de servicios, ingresos y rendimiento.

1.6.3 Servicios de inteligencia artificial (integrados en el backend)

1. Procesamiento de audio

- Conversión de audio a texto.
- Extracción de palabras clave sobre el incidente.

2. Clasificación preliminar del incidente

- Identificación aproximada del problema reportado.
- Categorización en clases como batería, llanta, choque, motor, suspensión, sistema eléctrico u otros.

3. Análisis básico de imágenes

- Detección de daños visibles.
- Apoyo a la clasificación del incidente.

4. Generación de resumen estructurado

- Construcción de una ficha sintética del caso para ayudar al taller en la toma de decisiones.

5. Sistema de asignación inteligente

- Evaluación de cercanía, especialidad, disponibilidad, capacidad operativa y prioridad.
- Generación de lista de talleres candidatos.
- Selección del taller óptimo.

1.7 Sistema de información basado en computadoras

1.7.1 Hardware

Servidor

Infraestructura en la nube:

- AWS para hosting principal del backend y servicios complementarios.
- Servicio administrado para la API backend.
- Base de datos PostgreSQL en servicio administrado.
- Almacenamiento de evidencias en servicio de objetos.

Cliente

Para administración y operación del taller (Web):

- Computadoras o laptops con navegador moderno.
- Resolución mínima recomendada: 1366x768.
- Sistema operativo: Windows, Linux o macOS.

Para clientes (Móvil):

- Smartphones Android o iOS.
- GPS activo para geolocalización.
- Cámara y micrófono para captura de evidencias.
- Conectividad móvil o Wi-Fi.

Medios de comunicación

- Conexión a internet estable para uso del panel web y la app móvil.
- HTTPS/TLS para intercambio seguro de datos.
- Servicios de notificación push para alertas en tiempo real.

Otros (opcional)

- Impresora para comprobantes internos o reportes.
- Equipos móviles del técnico para coordinación operativa.
- UPS o respaldo eléctrico para equipos administrativos del taller.

1.7.2 Software

Servidor

Componente	Tecnología / Versión
Backend	Python 3.11+ con FastAPI
API REST	FastAPI
Base de datos	PostgreSQL 16+
Sistema operativo	Linux (Ubuntu Server 22.04 LTS)
Servidor de aplicaciones	Uvicorn / Gunicorn
Autenticación	JWT
Seguridad	HTTPS (TLS 1.3), hash de contraseñas y control de acceso por roles
Almacenamiento	AWS S3 o servicio equivalente
Notificaciones	Firebase Cloud Messaging (FCM)
Contenedores	Docker 24+ / Docker Compose 2+
Control de versiones	Git 2.40+

Ciente Web

Componente	Tecnología / Versión
Framework	Angular 20+
Lenguaje	TypeScript 5+
UI / componentes	PrimeNG / Angular Material
Estado / servicios	Servicios Angular y consumo REST
Mapas	Leaflet o Google Maps
Navegadores soportados	Chrome, Edge, Firefox, Safari
Sistema operativo	Windows, Linux, macOS
Despliegue	Hosting estático o CDN

Ciente Móvil

Componente	Tecnología / Versión
Framework	Flutter 3.9+
Lenguaje	Dart 3.9+
Plataformas	Android 7.0+, iOS 12.0+
Geolocalización	geolocator / Google Maps
Multimedia	image_picker, camera, audio recorder

Notificaciones Push	firebase_messaging
HTTP Client	dio / http
Almacenamiento local	shared_preferences / SQLite

Herramientas de Desarrollo

Componente	Tecnología / Versión
Backend IDE	VS Code / PyCharm
Frontend IDE	VS Code / WebStorm
Móvil IDE	VS Code / Android Studio
Diseño UI/UX	Figma
Pruebas API	Postman / Insomnia
Gestión de proyecto	Trello / Jira / GitHub Projects
Documentación	Google Docs / Markdown
Control de versiones	Git / GitHub

1.7.3 Datos

El sistema manejará principalmente los siguientes grupos de datos:

- Datos de usuarios: nombre, contacto, credenciales, rol y estado.
- Datos de vehículos: placa, marca, modelo, color, año y propietario.
- Datos de talleres: nombre, ubicación, especialidades, reputación y horario.
- Datos de técnicos: identificación, especialidad, estado y disponibilidad.

- Datos de incidentes: tipo, descripción, prioridad, estado, ubicación, fecha y hora.
- Evidencias: imágenes, audio, texto y metadatos asociados.
- Historial de estados: cambios durante el ciclo de atención.
- Datos de pago.
- Calificaciones y comentarios del cliente.

1.7.4 Procesos

Procesos operativos:

- Registro de clientes y vehículos.
- Registro de emergencias.
- Recepción y evaluación de solicitudes.
- Aceptación y asignación del servicio.
- Seguimiento del estado de atención.
- Cierre del servicio y pago.

Procesos inteligentes:

- Transcripción de audio.
- Clasificación de incidentes.
- Resumen automático.
- Priorización del caso.
- Selección de talleres candidatos.

Procesos de soporte:

- Autenticación y autorización.
- Gestión de notificaciones.
- Auditoría de acciones.
- Monitoreo técnico y respaldo de información.

1.7.5 Gente

Los roles principales del sistema serán:

- Cliente: registra vehículos, reporta emergencias, da seguimiento y realiza pagos.

- Administrador del taller: recibe solicitudes, decide aceptación, gestiona técnicos y controla operaciones.
- Técnico (recurso operativo): recurso operativo del taller asignado a atenciones cuando corresponda; no interactúa directamente con el sistema en el MVP.

1.7.6 Documentación

La documentación del sistema contemplará:

- Manual de usuario para clientes.
- Manual de usuario para talleres.
- Documentación técnica del backend y APIs.
- Diagramas UML del sistema.
- Manual básico de despliegue y operación.

1.8 Tecnología

Lenguajes de programación:

- Python para backend.
- TypeScript para frontend Angular.
- Dart para Flutter.
- SQL para consultas y gestión relacional.

Frameworks y librerías:

- FastAPI para APIs REST.
- Angular para la aplicación web.
- Flutter para la aplicación móvil.
- PostgreSQL como gestor de base de datos.
- Librerías de mapas y geolocalización.
- Librerías de carga de imágenes, grabación de audio y push notifications.

Infraestructura y despliegue:

- AWS S3 para almacenamiento de evidencias.

- AWS RDS para PostgreSQL.
- AWS App Runner o servicio equivalente para backend.
- Hosting estático para frontend web.

Seguridad:

- JWT para autenticación.
- Cifrado de contraseñas con bcrypt.
- HTTPS/TLS para comunicación segura.
- Control de acceso por roles.
- Validación y sanitización de entradas.

Herramientas de productividad:

- Git y GitHub.
- Trello o Jira para gestión del proyecto.
- Figma para prototipos.
- Google Workspace para documentación.

1.9 Costos estimados

Los siguientes costos son referenciales y responden a un escenario de prototipo académico funcional desplegado en la nube.

Costos de desarrollo:

- Análisis y diseño del sistema: 400 USD.
- Desarrollo backend FastAPI: 800 USD.
- Desarrollo frontend Angular: 700 USD.
- Desarrollo app Flutter: 800 USD.
- Modelado, pruebas y documentación: 500 USD.

Costo estimado inicial: 3,200 USD.

Costos operativos mensuales aproximados:

- Infraestructura cloud básica: 80 a 150 USD/mes.

- Almacenamiento de evidencias: 10 a 30 USD/mes.
- Servicio de notificaciones y procesamiento adicional: 0 a 25 USD/mes.
- Dominio y certificados: costo bajo o gratuito según entorno.

Nota: Estos valores son aproximados y pueden variar según la cantidad de usuarios, volumen de evidencias cargadas, frecuencia de uso y servicios de inteligencia artificial utilizados.

1.10 Beneficios

Beneficios operativos:

- Reducción del tiempo de atención ante emergencias.
- Mejor asignación de talleres y técnicos.
- Mayor organización y trazabilidad del servicio.

Beneficios para el cliente:

- Canal único y estructurado para solicitar auxilio.
- Seguimiento en tiempo real del estado de la atención.
- Mayor confianza al trabajar con talleres identificados y evaluables.

Beneficios para los talleres:

- • Recepción de solicitudes mejor clasificadas.
- • Optimización de recursos y disponibilidad.
- • Historial centralizado de atenciones y mejor control operativo.

Beneficios estratégicos:

- Integración de IA dentro del flujo principal del negocio.
- Posibilidad de crecimiento hacia un ecosistema mayor de asistencia vehicular.
- Base tecnológica sólida para futuras expansiones como remolque avanzado, seguros, analítica y reputación inteligente.

CAPÍTULO 2: FUNDAMENTACIÓN TEÓRICA

2.1 Concepto de auxilio mecánico vehicular

El auxilio mecánico vehicular es el conjunto de servicios orientados a asistir a un conductor cuando su vehículo presenta una avería, incidente o condición que impide su funcionamiento normal o compromete la seguridad de la circulación. Este servicio puede involucrar atención en sitio, diagnóstico preliminar, reparación básica, suministro de energía, cambio de llanta, apertura de vehículo, remolque o derivación a un taller especializado.

Desde la perspectiva de sistemas de información, el auxilio mecánico no debe entenderse únicamente como una actividad técnica, sino como un proceso coordinado de captura de información, evaluación del incidente, asignación de recursos, ejecución operativa y cierre del servicio. Por ello, su digitalización exige integrar actores, datos, reglas de negocio, estados del proceso y mecanismos de comunicación en tiempo real.

2.2 Importancia de la digitalización del servicio

La digitalización del auxilio mecánico permite transformar un servicio tradicionalmente reactivo, informal y poco trazable en un proceso estructurado y optimizable. Entre sus ventajas se encuentran:

- mejor calidad de información al momento de reportar el incidente;
- reducción del tiempo de respuesta mediante geolocalización;
- asignación más precisa de talleres y técnicos;
- trazabilidad completa del servicio;
- mayor confianza del cliente gracias al seguimiento y reputación del proveedor.

Además, la incorporación de procesamiento multimodal —texto, audio, imagen y ubicación— permite enriquecer el diagnóstico preliminar y disminuir la incertidumbre inicial, algo especialmente importante cuando el cliente no tiene conocimientos mecánicos suficientes para describir adecuadamente la falla.

2.3 Componentes clave del servicio

Para que una plataforma de atención de emergencias vehiculares funcione de forma adecuada, deben integrarse al menos los siguientes componentes:

1. Captura de la solicitud

Permite que el cliente registre la emergencia con datos del vehículo, descripción del problema, evidencia visual, evidencia auditiva y ubicación geográfica.

2. Clasificación preliminar

Utiliza reglas de negocio y servicios inteligentes para aproximar el tipo de incidente, su prioridad y la posible necesidad de remolque o atención especializada.

3. Motor de selección y asignación

Evalúa talleres candidatos considerando cercanía, especialidad, disponibilidad, capacidad operativa y prioridad del caso.

4. Operación del taller

Incluye aceptación o rechazo de la solicitud, asignación de técnico, actualización de estados y control del tiempo de atención.

5. Seguimiento y cierre

Abarca la visualización del estado del servicio, el registro del pago, la calificación del cliente y la conservación del historial.

2.4 Flujo operativo general del servicio

El flujo operativo general propuesto para la plataforma se describe en las siguientes etapas:

1. El cliente inicia sesión y selecciona el vehículo afectado.
2. Registra la emergencia mediante texto, audio, fotografía y ubicación.
3. El backend procesa la evidencia y genera una clasificación preliminar.
4. El sistema calcula prioridad y construye una lista de talleres candidatos.
5. Se notifica a los talleres más adecuados.
6. Un taller acepta la solicitud y asigna un técnico disponible.
7. El cliente visualiza el estado de atención y el tiempo estimado.
8. El técnico atiende el incidente y el taller actualiza el estado del servicio.
9. El sistema registra el cierre, el pago y la calificación.

2.5 Retos funcionales del dominio

La gestión del auxilio vehicular presenta desafíos particulares que deben ser resueltos por el sistema:

- incertidumbre en la descripción del problema por parte del cliente;
- necesidad de respuesta rápida en contextos de urgencia;
- alta dependencia de la ubicación geográfica;
- variabilidad en especialidades técnicas requeridas;
- necesidad de coordinación entre múltiples actores;
- importancia de mantener comunicación continua con el usuario.

Estos retos justifican el uso de una arquitectura orientada a servicios, modelos de datos trazables y mecanismos inteligentes de clasificación y asignación.

2.6 Referentes funcionales y estado del arte

2.6.1 Plataformas de intermediación bajo demanda

Las plataformas bajo demanda han demostrado que es posible conectar usuarios con proveedores de servicio de manera rápida y estructurada. Su principal aporte funcional consiste en centralizar la solicitud, geolocalizar al usuario, seleccionar proveedores disponibles, mostrar estados del servicio y mantener la trazabilidad del proceso. Aunque este modelo ha sido ampliamente adoptado en transporte y delivery, sus principios también resultan válidos para la atención de emergencias vehiculares.

De estos referentes se pueden rescatar varios aprendizajes: la importancia de la experiencia de usuario en el registro de la solicitud, la necesidad de mostrar estados claros, la conveniencia de integrar reputación de proveedores y la utilidad de los sistemas de notificación en tiempo real.

2.6.2 Servicios tradicionales de asistencia vial

Los servicios tradicionales de asistencia vial suelen ofrecer cobertura para remolque, paso de corriente, cambio de llanta o apoyo ante incidentes mecánicos. No obstante, muchas veces operan mediante call centers, redes de proveedores y procesos poco transparentes desde la perspectiva del usuario. Si bien cumplen una función importante, normalmente no permiten que el cliente adjunte evidencia multimodal ni visualice de forma detallada el proceso interno de asignación.

Frente a ello, la plataforma propuesta agrega valor al digitalizar el registro del incidente, enriquecer el diagnóstico preliminar y mejorar la trazabilidad del proceso.

2.6.3 Directorios digitales y marketplaces de talleres

Otro referente lo constituyen los directorios y marketplaces de talleres, donde el usuario busca manualmente un proveedor según ubicación o especialidad. El aporte de estos sistemas es que organizan la oferta de servicios y permiten visualizar información básica de los talleres. Sin embargo, el problema principal es que el usuario todavía debe interpretar por sí mismo el incidente, comparar opciones y coordinar la atención sin apoyo inteligente.

La propuesta académica de este proyecto va un paso más allá al integrar un motor de clasificación y asignación que disminuye la carga de decisión del usuario en un contexto de emergencia.

2.6.4 Posicionamiento de la propuesta

La plataforma propuesta se posiciona como una solución híbrida entre aplicación de intermediación, sistema operativo para talleres y servicio inteligente de apoyo a la decisión. Su diferenciación principal se fundamenta en cuatro aspectos:

- registro multimodal de la emergencia;
- clasificación preliminar apoyada por IA;
- asignación inteligente de talleres y técnicos;
- trazabilidad completa del servicio hasta su cierre.

Este posicionamiento convierte al proyecto en una propuesta con valor académico y potencial de aplicación real, especialmente en contextos donde el servicio todavía se presta de forma fragmentada e informal.

2.7 Uso de la inteligencia artificial en la plataforma

2.7.1 Rol de la inteligencia artificial dentro del sistema

La inteligencia artificial en esta propuesta no se concibe como un complemento aislado, sino como un componente transversal que apoya la toma de decisiones dentro del flujo principal del servicio. Su función no es reemplazar el criterio técnico del taller, sino reducir la incertidumbre, enriquecer la información del caso y mejorar el proceso de asignación.

2.7.2 Procesamiento de audio

El sistema puede utilizar servicios de transcripción de voz para convertir la explicación oral del cliente en texto estructurado. Esto aporta dos beneficios directos:

- permite capturar información de manera rápida incluso si el cliente no desea escribir;
- facilita el análisis posterior de palabras clave asociadas a incidentes como batería, llanta, humo, choque, motor o grúa.

2.7.3 Clasificación preliminar de incidentes

La clasificación preliminar busca determinar una categoría aproximada del incidente usando una combinación de texto, transcripción de audio, imágenes y reglas de negocio. Las categorías iniciales recomendadas para el proyecto son:

- batería
- llanta
- sistema eléctrico
- motor
- suspensión
- choque leve
- remolque
- otro / incierto

Cuando el sistema detecte baja confianza o datos insuficientes, deberá marcar el incidente como incierto y permitir revisión manual del taller.

2.7.4 Análisis básico de imágenes

El análisis de imágenes puede ayudar a detectar daños visibles o reforzar la clasificación del incidente. Para el contexto académico del proyecto, no es necesario construir un sistema de visión artificial complejo; basta con integrar una lógica básica de apoyo que complemente la información proporcionada por el usuario. Su propósito principal será enriquecer el resumen del caso y apoyar la toma de decisiones del taller.

2.7.5 Generación de resumen del incidente

La plataforma debe construir automáticamente una ficha estructurada que concentre la información más relevante del caso. Por ejemplo:

- vehículo afectado;
- ubicación;
- descripción resumida;
- clasificación preliminar;
- prioridad estimada;
- necesidad de remolque;
- observaciones relevantes extraídas de audio, texto o imagen.

Este resumen reduce el tiempo que el taller necesita para interpretar la solicitud y mejora la calidad de la decisión de aceptación.

2.7.6 Sistema de priorización y asignación

La IA también puede intervenir en la priorización del caso y en la selección de talleres candidatos. La prioridad puede calcularse considerando:

- tipo de incidente;
- contexto reportado por el cliente;
- posibilidad de movilidad del vehículo;
- disponibilidad de recursos;
- distancia entre cliente y proveedor.

El motor de asignación debe combinar criterios técnicos y geográficos para proponer una lista de talleres compatibles, ordenados por nivel de conveniencia.

2.7.7 Beneficios del enfoque inteligente

La incorporación de IA permite:

- mejorar el diagnóstico preliminar;
- reducir tiempos de evaluación;
- elevar la calidad de la asignación;
- mejorar la experiencia del usuario;
- fortalecer la eficiencia operativa del taller.

2.8 Herramientas tecnológicas

2.8.1 FastAPI

FastAPI es una alternativa adecuada para el backend del proyecto porque permite construir APIs REST modernas con alto rendimiento, validación automática de datos, documentación interactiva y una estructura clara para el desarrollo de servicios. Su enfoque resulta conveniente para una plataforma basada en múltiples módulos y procesos en tiempo real.

2.8.2 Angular

Angular resulta apropiado para la aplicación web orientada a talleres porque facilita la construcción de interfaces administrativas complejas, manejo de formularios, tablas, rutas protegidas y consumo estructurado de APIs. También permite desarrollar paneles con componentes reutilizables y vistas organizadas por módulos funcionales.

2.8.3 Flutter

Flutter es una opción conveniente para la aplicación móvil del cliente debido a su capacidad multiplataforma, su velocidad de desarrollo y su integración con funcionalidades nativas como cámara, geolocalización, grabación de audio y notificaciones push. Esto lo convierte en una herramienta sólida para una app centrada en registro de incidentes y seguimiento del servicio.

2.8.4 PostgreSQL

PostgreSQL ofrece una base de datos robusta, relacional y adecuada para manejar trazabilidad, estados, relaciones entre usuarios, talleres, técnicos, vehículos, incidentes y evidencias. Su uso es coherente con los requerimientos del sistema,

especialmente por la necesidad de mantener integridad de datos y consultas estructuradas.

2.8.5 AWS

AWS proporciona servicios adecuados para alojar la solución y distribuir responsabilidades entre backend, base de datos, almacenamiento de evidencias y despliegue de frontend. Su adopción también fortalece el componente académico del proyecto al introducir prácticas de despliegue y arquitectura cloud.

2.8.6 GitHub y control de versiones

GitHub permite gestionar el código fuente, trabajar colaborativamente, mantener historial de cambios y estructurar el desarrollo por ramas, issues y milestones. También facilita la documentación del proyecto y la futura automatización de despliegues.

2.8.7 Herramientas complementarias

Como herramientas de apoyo se recomienda utilizar:

- Figma para prototipos y diseño de interfaces;
- Postman o Insomnia para pruebas de API;
- Trello o Jira para organización del trabajo;
- Google Workspace para documentación colaborativa.

2.9 Proceso Unificado de Desarrollo de Software (PUDS)

2.9.1 Concepto general

El Proceso Unificado de Desarrollo de Software es una metodología iterativa e incremental orientada a objetos, centrada en casos de uso y en la arquitectura del sistema. Su adopción en este proyecto permite organizar el trabajo en fases, reducir riesgos y construir el sistema de manera progresiva.

2.9.2 Fases del PUDS

Inicio:

Se define la visión general del proyecto, el problema, los objetivos, el alcance, los actores y la factibilidad general de la solución.

Elaboración:

Se profundiza en los requisitos, se definen los casos de uso críticos, se diseña la arquitectura base y se mitigan los riesgos técnicos más importantes.

Construcción:

Se implementan los módulos del sistema de manera iterativa, incorporando pruebas continuas y refinamiento funcional.

Transición:

Se despliega la solución, se corrigen defectos finales, se documenta el sistema y se prepara la defensa o presentación del proyecto.

2.9.3 Pertinencia del PUDS para el proyecto

El PUDS resulta adecuado para esta propuesta porque:

- permite trabajar por iteraciones y priorizar funcionalidades críticas;
- facilita la organización del sistema alrededor de casos de uso;
- promueve una arquitectura definida desde etapas tempranas;
- encaja bien con el modelado UML exigido en la materia.

2.10 UML

2.10.1 Definición

El Lenguaje Unificado de Modelado (UML) es un estándar de modelado visual utilizado para representar la estructura y el comportamiento de sistemas de software. En el contexto de este proyecto, UML servirá para documentar actores, flujos, clases, componentes y despliegue.

2.10.2 Propósitos de UML en el proyecto

El uso de UML permitirá:

- comunicar de forma clara la estructura del sistema;
- visualizar el flujo de interacción entre actores y módulos;
- documentar el diseño de clases y relaciones de datos;
- representar la arquitectura lógica y física de la solución.

2.10.3 Diagramas recomendados para el proyecto

Para este sistema se consideran especialmente pertinentes los siguientes diagramas:

- diagrama de casos de uso;
- diagrama de actividad;
- diagrama de secuencia;
- diagrama de clases;
- diagrama de componentes;
- diagrama de despliegue.

2.10.4 Importancia académica y técnica

El modelado UML aporta trazabilidad entre requisitos, análisis y diseño. Además, permite justificar decisiones técnicas y mostrar de forma ordenada la evolución del sistema desde la idea inicial hasta su implementación.

CAPÍTULO 3: PROCESO DE DESARROLLO DEL SOFTWARE

3.1 FT: CAPTURA DE REQUISITOS

3.1.1 Identificar actores

ID	ACTOR	DESCRIPCIÓN
A1	Cliente	Usuario que registra vehículos, reporta emergencias, adjunta evidencias, realiza pagos y califica el servicio.
A2	Administrador de Taller	Responsable de gestionar las solicitudes recibidas, aceptar o rechazar incidentes, asignar técnicos y supervisar la atención.
A3	Servicio de Notificaciones	Servicio externo utilizado para distribuir alertas y actualizaciones en tiempo real a clientes y talleres.
A4	Servicio de Rutas y	Servicio externo utilizado

	Geolocalización	para apoyar la ubicación del incidente, estimar distancias y, cuando corresponda, calcular rutas.
A5	Pasarela de Pago	Servicio externo utilizado para registrar o procesar pagos del servicio cuando se use integración externa.

3.1.2 Identificar casos de uso

ID	NOMBRE	ENTORNO
CU01	Registrar usuario	Móvil / Web
CU02	Iniciar sesión	Móvil / Web
CU03	Registrar vehículo	Móvil
CU04	Reportar emergencia	Móvil
CU05	Adjuntar evidencias	Móvil
CU06	Consultar estado de solicitud	Móvil
CU07	Recibir notificaciones push del servicio	Móvil / Web
CU08	Gestionar información del taller	Web
CU09	Gestionar técnicos	Web
CU10	Gestionar disponibilidad	Web
CU11	Visualizar solicitudes disponibles	Web
CU12	Aceptar o rechazar solicitud	Web
CU13	Asignar técnico	Web
CU14	Gestionar atención del servicio	Web

CU15	Generar resumen del incidente	Backend
CU16	Analizar evidencias del incidente	Backend
CU17	Clasificar y priorizar incidente	Backend
CU18	Seleccionar talleres candidatos	Backend
CU19	Registrar pago	Móvil
CU20	Calificar servicio	Móvil
CU21	Consultar historial de atenciones	Móvil / Web
CU22	Gestionar métricas y reportes	Web

3.1.3 Priorización de casos de uso

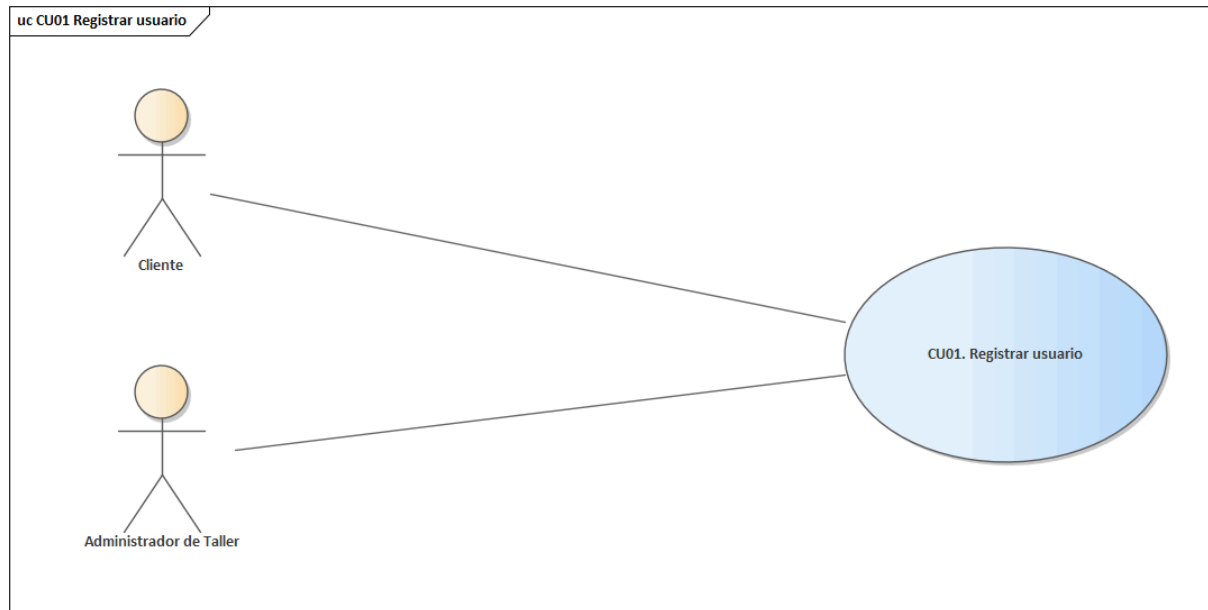
ITERACIÓN	ID	CASO DE USO	PRIORIDAD
Ciclo 1	CU01	Registrar usuario	Alta
Ciclo 1	CU02	Iniciar sesión	Alta
Ciclo 1	CU03	Registrar vehículo	Alta
Ciclo 1	CU04	Reportar emergencia	Alta
Ciclo 1	CU05	Adjuntar evidencias	Alta
Ciclo 2	CU06	Consultar estado de solicitud	Media
Ciclo 2	CU07	Recibir notificaciones push del servicio	Media

Ciclo 1	CU08	Gestionar información del taller	Alta
Ciclo 1	CU09	Gestionar técnicos	Alta
Ciclo 2	CU10	Gestionar disponibilidad	Media
Ciclo 1	CU11	Visualizar solicitudes disponibles	Alta
Ciclo 1	CU12	Aceptar o rechazar solicitud	Alta
Ciclo 1	CU13	Asignar técnico	Alta
Ciclo 1	CU14	Gestionar atención del servicio	Alta
Ciclo 2	CU15	Generar resumen del incidente	Media
Ciclo 2	CU16	Analizar evidencias del incidente	Media
Ciclo 2	CU17	Clasificar y priorizar incidente	Media
Ciclo 2	CU18	Seleccionar talleres candidatos	Media
Ciclo 3	CU19	Registrar pago	Baja
Ciclo 3	CU20	Calificar servicio	Baja
Ciclo 3	CU21	Consultar historial de atenciones	Baja
Ciclo 3	CU22	Gestionar métricas y reportes	Baja

3.1.4 Especificar casos de uso

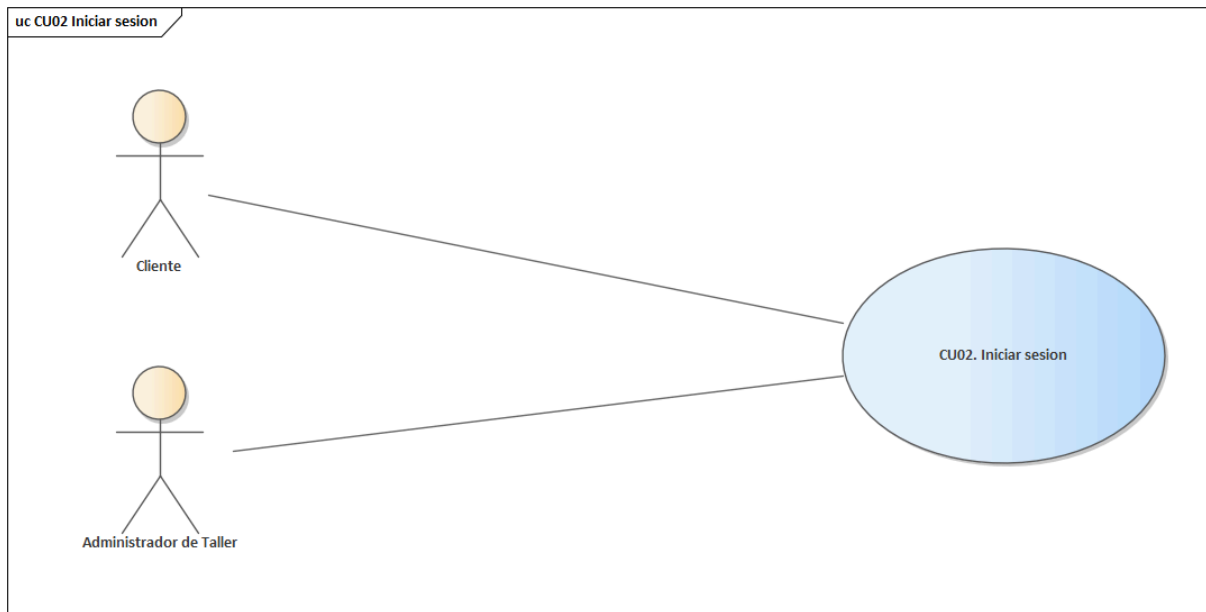
Ciclo #1

CU01 Registrar usuario



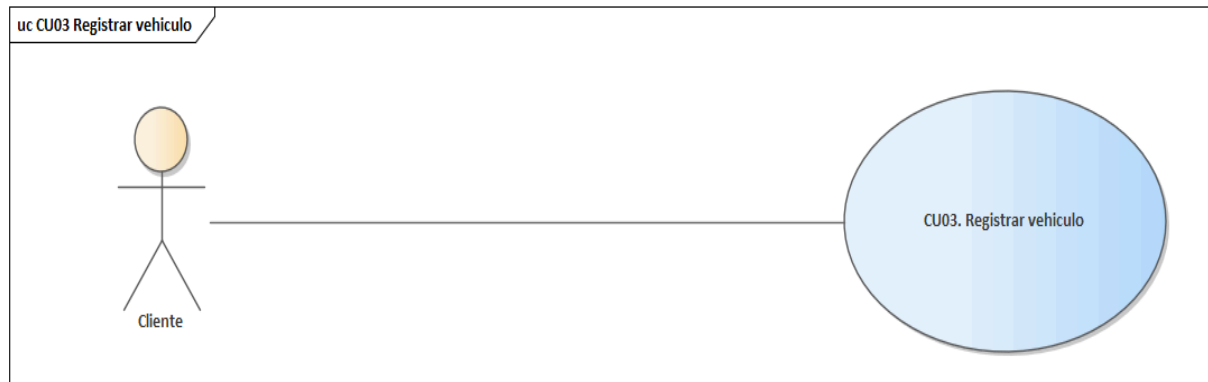
CAMPO	DETALLE
Nombre del caso de uso	CU01 Registrar usuario
Propósito	Permitir la creación de una cuenta para acceder al sistema desde el entorno correspondiente.
Actores	Cliente; Administrador de Taller
Actor iniciador	Usuario nuevo
Precondición	No aplica, salvo disponibilidad del formulario o canal de registro.
Flujo principal	1) El usuario accede al formulario de registro. 2) Completa sus datos básicos. 3) Define credenciales. 4) El sistema valida duplicados y consistencia. 5) Se almacena el nuevo usuario.
Postcondición	La cuenta queda creada y disponible para autenticación posterior.
Excepciones	Correo ya registrado; contraseñas inconsistentes; datos obligatorios incompletos.

CU02 Iniciar sesión



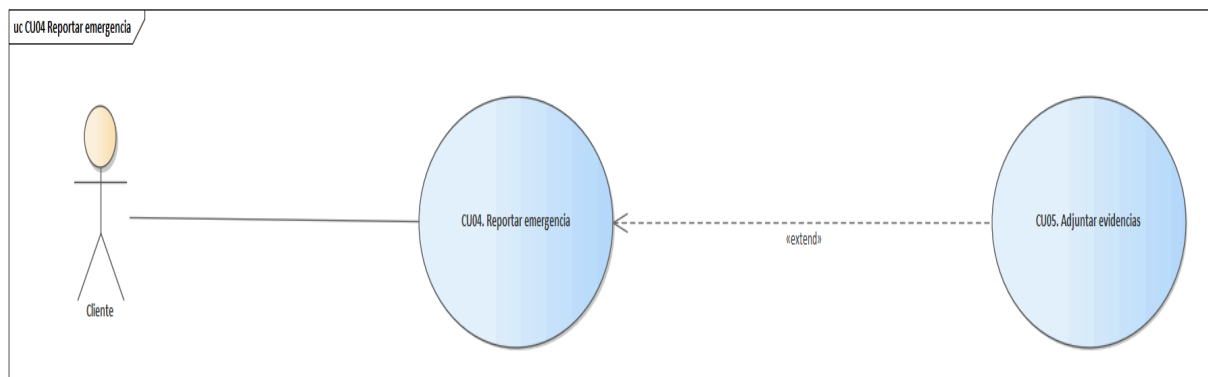
CAMPO	DETALLE
Nombre del caso de uso	CU02 Iniciar sesión
Propósito	Permitir que un usuario autenticado acceda al sistema según el rol que le corresponda.
Actores	Cliente; Administrador de Taller
Actor iniciador	Usuario
Precondición	El usuario debe encontrarse previamente registrado.
Flujo principal	1) El usuario accede a la pantalla de autenticación. 2) Ingresa correo y contraseña. 3) El sistema valida credenciales. 4) Se genera la sesión o token correspondiente. 5) Se redirige al entorno autorizado.
Postcondición	La sesión queda iniciada y el usuario puede acceder a las funciones según su rol.
Excepciones	Credenciales inválidas; usuario inactivo; token expirado o sesión no autorizada.

CU03 Registrar vehículo



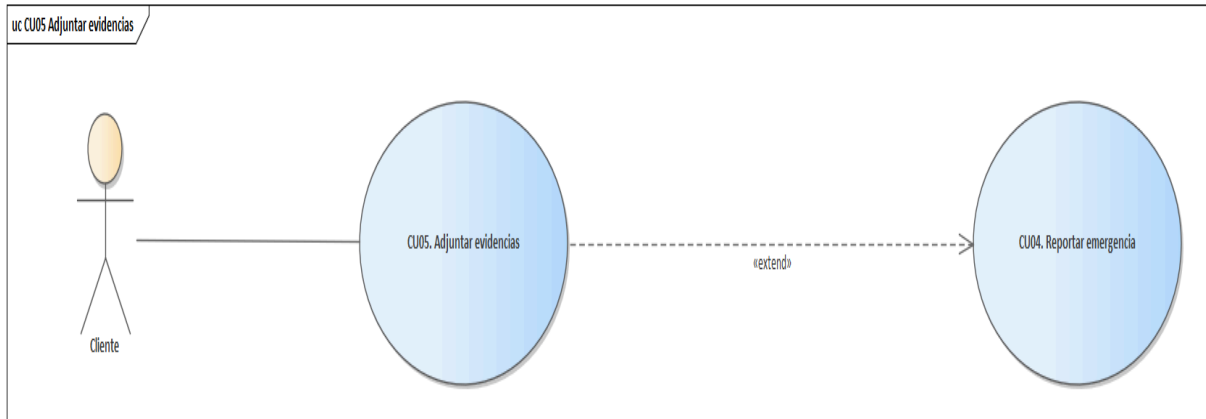
CAMPO	DETALLE
Nombre del caso de uso	CU03 Registrar vehículo
Propósito	Permitir que el cliente registre uno o varios vehículos dentro de su cuenta.
Actores	Cliente
Actor iniciador	Cliente
Precondición	El cliente debe haber iniciado sesión.
Flujo principal	1) El cliente accede al módulo de vehículos. 2) Ingresa placa, marca, modelo, año y color. 3) Confirma el registro. 4) El sistema valida la información y almacena el vehículo asociado al cliente.
Postcondición	El vehículo queda registrado y disponible para asociarse a futuras emergencias.
Excepciones	Datos incompletos; placa duplicada; error de validación en formato de campos.

CU04 Reportar emergencia



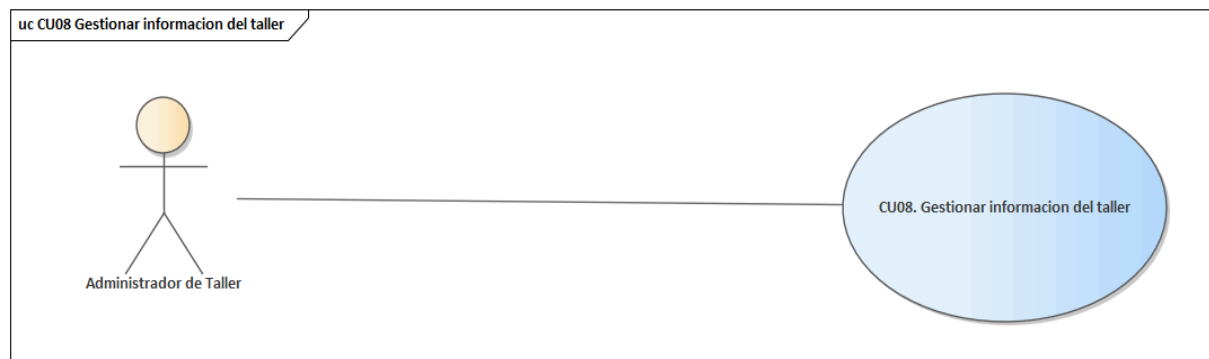
CAMPO	DETALLE
Nombre del caso de uso	CU04 Reportar emergencia
Propósito	Permitir que el cliente registre un incidente vehicular con información suficiente para su análisis y atención.
Actores	Cliente
Actor iniciador	Cliente
Precondición	El cliente debe haber iniciado sesión y tener al menos un vehículo registrado.
Flujo principal	1) El cliente accede al módulo de emergencias. 2) Selecciona el vehículo. 3) Permite el uso de ubicación. 4) Adjunta evidencia. 5) Confirma el registro. 6) El sistema almacena la solicitud.
Postcondición	La emergencia queda registrada y lista para análisis.
Excepciones	Falta de conexión; ausencia de ubicación; datos incompletos.

CU05 Adjuntar evidencias



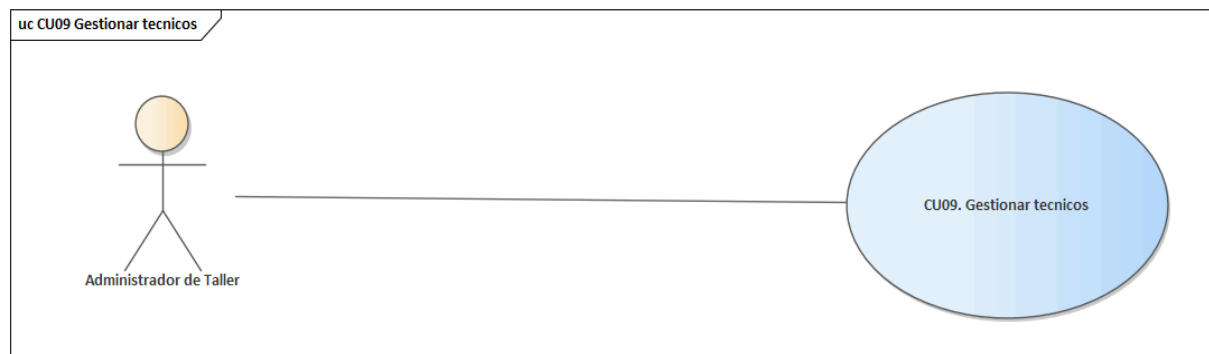
CAMPO	DETALLE
Nombre del caso de uso	CU05 Adjuntar evidencias
Propósito	Permitir que el cliente adjunte imágenes, audio o texto complementario para enriquecer la descripción del incidente.
Actores	Cliente
Actor iniciador	Cliente
Precondición	Debe existir un incidente en proceso de registro o ya registrado por el cliente.
Flujo principal	1) El cliente accede a la opción de adjuntar evidencias. 2) Selecciona si desea cargar imagen, audio o texto. 3) El sistema solicita permisos o acceso al recurso correspondiente. 4) El cliente adjunta el archivo o escribe la información. 5) El sistema valida el formato y almacena la evidencia asociada al incidente.
Postcondición	Las evidencias quedan registradas y asociadas al incidente para su posterior análisis.
Excepciones	Archivo no soportado; error de carga; permisos denegados; tamaño excedido.

CU08 Gestionar información del taller



CAMPO	DETALLE
Nombre del caso de uso	CU08 Gestionar información del taller
Propósito	Permitir que el administrador registre y mantenga actualizada la información operativa del taller, su ubicación y especialidades.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	Debe existir un usuario con permisos para registrar el taller.
Flujo principal	1) El administrador accede al módulo del taller. 2) Registra o actualiza datos de identificación, ubicación y contacto. 3) Define especialidades o áreas de servicio. 4) El sistema valida y guarda la información.
Postcondición	La información del taller queda registrada o actualizada y disponible para la operación.
Excepciones	Datos obligatorios incompletos; duplicidad de taller; ubicación inválida o fuera de cobertura.

CU09 Gestionar técnicos



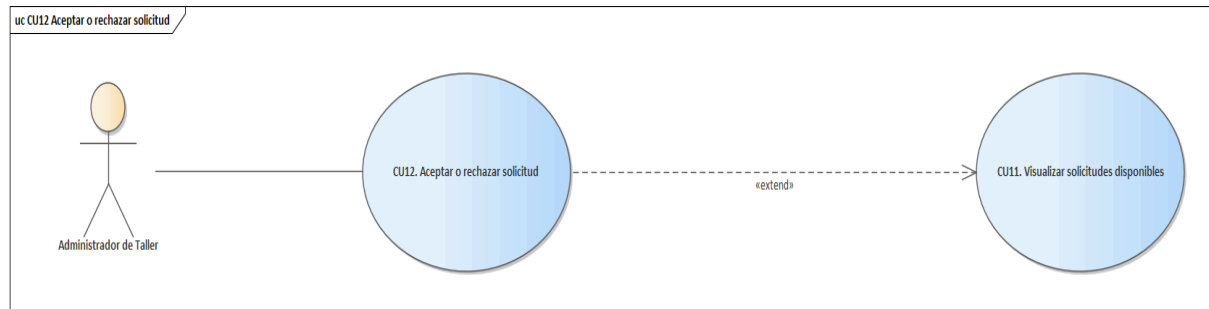
CAMPO	DETALLE
Nombre del caso de uso	CU09 Gestionar técnicos
Propósito	Permitir que el taller registre, actualice y administre técnicos o especialistas dentro de su estructura operativa.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	El taller debe encontrarse registrado y autenticado.
Flujo principal	1) El administrador accede al módulo de técnicos. 2) Registra datos personales y especialidad. 3) Define estado inicial de disponibilidad. 4) El sistema valida y almacena la información.
Postcondición	El técnico queda registrado y disponible para futuras asignaciones según su estado.
Excepciones	Datos incompletos; especialidad duplicada; error de validación de documento o contacto.

CU11 Visualizar solicitudes disponibles



CAMPO	DETALLE
Nombre del caso de uso	CU11 Visualizar solicitudes disponibles
Propósito	Permitir que el taller visualice los incidentes disponibles y compatibles con su capacidad operativa.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	El taller debe estar autenticado y activo dentro del sistema.
Flujo principal	1) El administrador del taller accede al módulo de solicitudes. 2) El sistema consulta incidentes compatibles. 3) Ordena por proximidad, prioridad y disponibilidad. 4) Presenta la lista para evaluación.
Postcondición	El taller visualiza las solicitudes candidatas disponibles para revisión.
Excepciones	No existen solicitudes vigentes; filtros sin coincidencias; error de conexión con backend.

CU12 Aceptar o rechazar solicitud



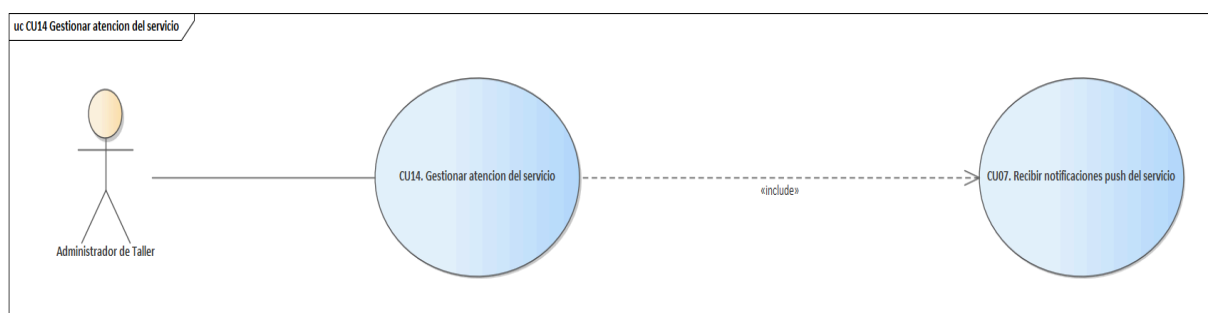
CAMPO	DETALLE
Nombre del caso de uso	CU12 Aceptar o rechazar solicitud
Propósito	Permitir que el taller decida si atenderá una solicitud disponible.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	Debe existir una solicitud activa y el taller debe tener sesión iniciada.
Flujo principal	1) El taller visualiza solicitudes disponibles. 2) Revisa el resumen. 3) Evalúa ubicación, problema y disponibilidad. 4) Acepta o rechaza. 5) El sistema registra la decisión.
Postcondición	La solicitud queda aceptada o permanece disponible para otros talleres candidatos.
Excepciones	Pérdida de disponibilidad; solicitud ya aceptada por otro taller.

CU13 Asignar técnico



CAMPO	DETALLE
Nombre del caso de uso	CU13 Asignar técnico
Propósito	Asignar un técnico disponible a la solicitud aceptada.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	La solicitud debe haber sido aceptada.
Flujo principal	1) El administrador visualiza los técnicos disponibles. 2) Selecciona el técnico compatible. 3) Confirma la asignación. 4) El sistema actualiza el estado del técnico y de la solicitud.
Postcondición	La solicitud queda asociada a un técnico.
Excepciones	No existen técnicos disponibles; conflicto de asignación.

CU14 Gestionar atención del servicio

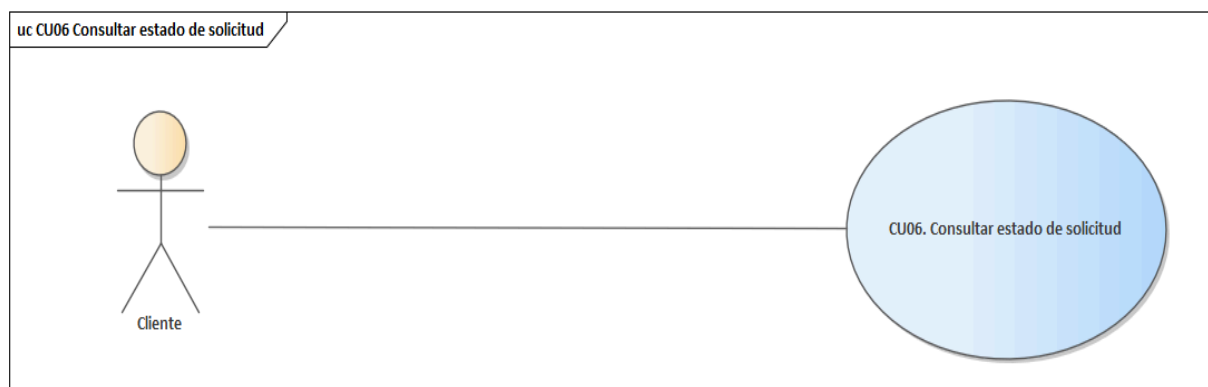


CAMPO	DETALLE
-------	---------

Nombre del caso de uso	CU14 Gestionar atención del servicio
Propósito	Gestionar el seguimiento operativo del servicio, registrando avances, observaciones y estados relevantes de la atención.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	Debe existir una solicitud activa y asignada.
Flujo principal	1) El administrador accede al detalle de la atención. 2) Revisa la información operativa y el técnico asignado. 3) Selecciona un nuevo estado o registra una observación. 4) El sistema guarda el cambio. 5) Se notifica al cliente cuando corresponda.
Postcondición	La atención queda actualizada con el nuevo estado registrado.
Excepciones	Falta de conexión; transición de estado no válida.

Ciclo #2

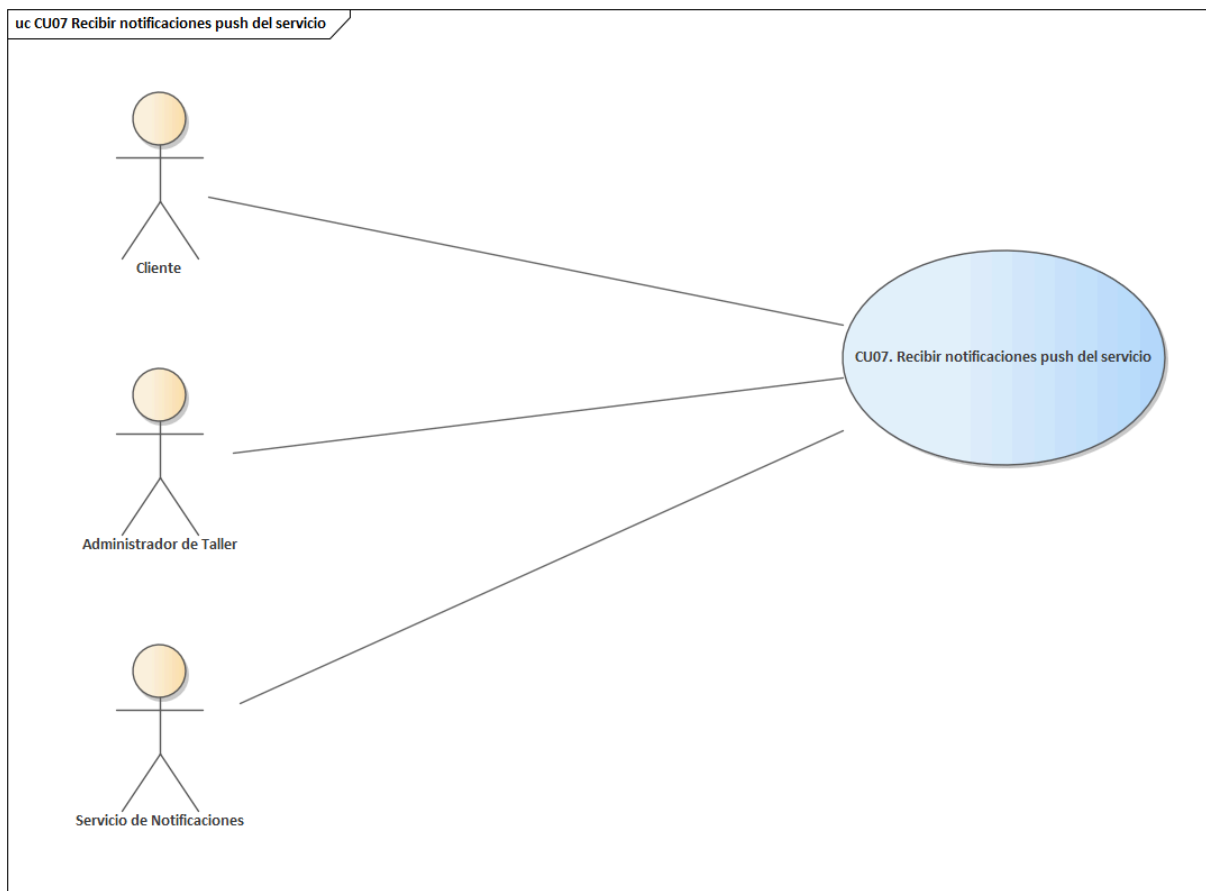
CU06 Consultar estado de solicitud



CAMPO	DETALLE
Nombre del caso de uso	CU06 Consultar estado de solicitud
Propósito	Permitir que el cliente consulte el estado actualizado de una solicitud o

	atención registrada.
Actores	Cliente
Actor iniciador	Cliente
Precondición	El cliente debe haber iniciado sesión y contar con al menos una solicitud registrada.
Flujo principal	1) El cliente ingresa a su historial o solicitud activa. 2) Selecciona un caso. 3) El sistema consulta el estado actual, taller asignado y observaciones. 4) Muestra la información resumida y actualizada.
Postcondición	El cliente visualiza el estado vigente y el historial básico del servicio.
Excepciones	Sin conexión; servicio inexistente; información aún no actualizada por el taller.

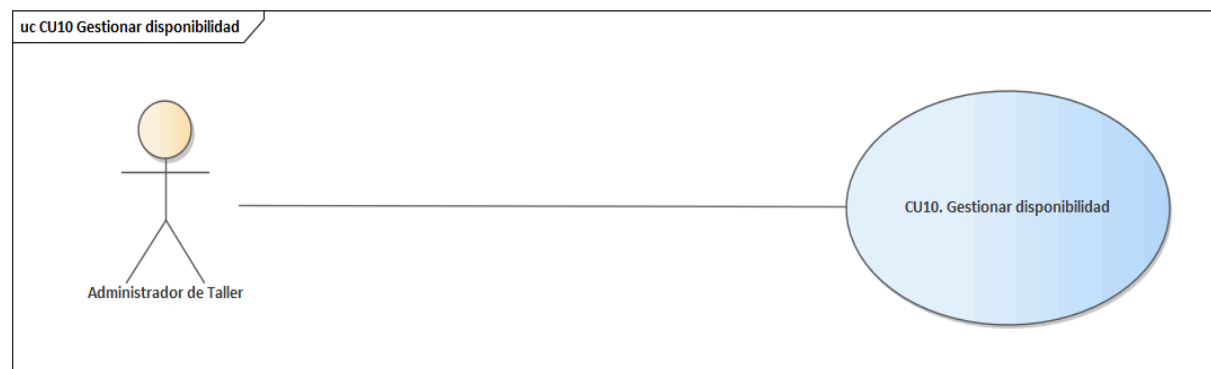
CU07 Recibir notificaciones push del servicio



CAMPO	DETALLE
Nombre del caso de uso	CU07 Recibir notificaciones push del servicio
Propósito	Permitir que clientes y talleres reciban alertas push sobre eventos relevantes del servicio, como aceptación, asignación, cambio de estado o confirmación de pago.
Actores	Cliente; Administrador de Taller; Servicio de Notificaciones
Actor iniciador	No aplica (proceso automático desencadenado por eventos del sistema)
Precondición	El usuario debe estar autenticado y con canal de notificación habilitado.
Flujo principal	1) El sistema detecta un evento relevante. 2) Determina los destinatarios

	correspondientes. 3) Genera el mensaje de notificación. 4) Envía la alerta al dispositivo o panel correspondiente. 5) El usuario visualiza la notificación recibida.
Postcondición	El usuario recibe una alerta asociada a un evento del sistema.
Excepciones	Canal no disponible; dispositivo sin conexión; notificación rechazada o no entregada.

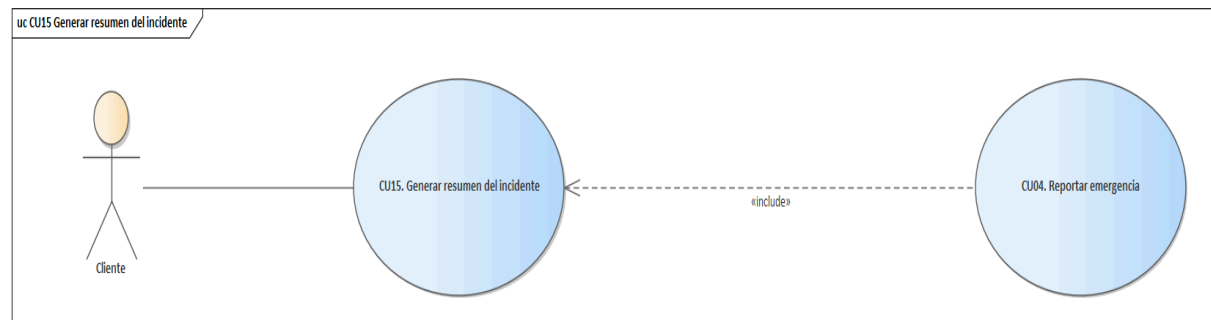
CU10 Gestionar disponibilidad



CAMPO	DETALLE
Nombre del caso de uso	CU10 Gestionar disponibilidad
Propósito	Gestionar la disponibilidad de técnicos y la capacidad operativa del taller.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	El taller debe contar con técnicos previamente registrados.
Flujo principal	1) El administrador accede al módulo de personal. 2) Revisa el estado actual de técnicos. 3) Activa o desactiva disponibilidad. 4) El sistema guarda la actualización y recalcula capacidad operativa.
Postcondición	La disponibilidad operativa del taller queda actualizada.

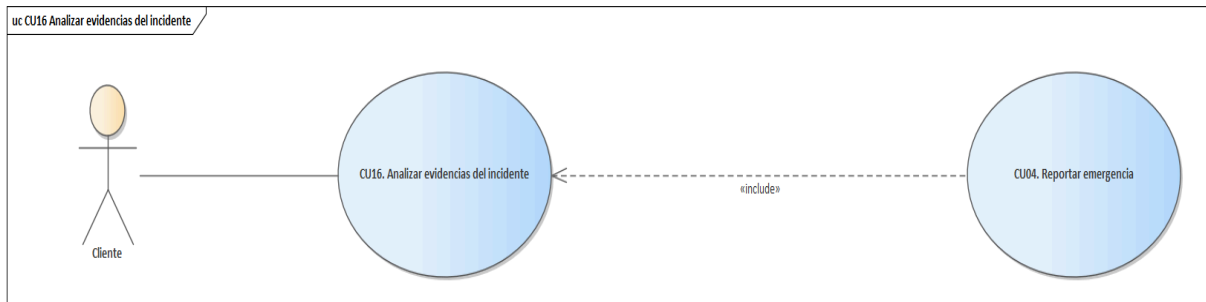
Excepciones	Disponibilidad inconsistente; técnico inexistente; conflicto con otra asignación activa.
-------------	--

CU15 Generar resumen del incidente



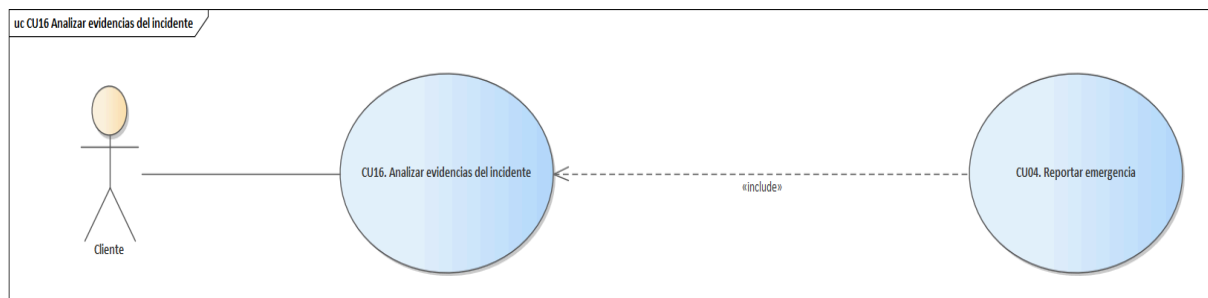
CAMPO	DETALLE
Nombre del caso de uso	CU15 Generar resumen del incidente
Propósito	Generar un resumen comprensible del incidente para facilitar su revisión y atención. Este caso de uso se ejecuta automáticamente como parte del procesamiento posterior al reporte de emergencia.
Actores	Cliente
Actor iniciador	Cliente
Precondición	Debe existir un incidente previamente registrado.
Flujo principal	1) El sistema recupera la información del incidente. 2) Analiza texto, audio transcrito y evidencias asociadas. 3) Sintetiza la información clave. 4) Genera un resumen breve y útil para talleres y operadores.
Postcondición	El incidente cuenta con un resumen estructurado para su visualización y análisis.
Excepciones	No existe evidencia suficiente; datos ambiguos; error en módulo de síntesis.

CU16 Analizar evidencias del incidente



CAMPO	DETALLE
Nombre del caso de uso	CU16 Analizar evidencias del incidente
Propósito	Analizar las evidencias del incidente, incluyendo texto, audio e imágenes, para extraer información útil que apoye la clasificación, priorización y asignación. Este caso de uso se ejecuta automáticamente como parte del procesamiento posterior al reporte de emergencia.
Actores	Cliente
Actor iniciador	Cliente
Precondición	Debe existir un incidente registrado con evidencias o descripción suficiente.
Flujo principal	1) El sistema recibe texto, audio o imagen del incidente. 2) Normaliza la información disponible. 3) Ejecuta reglas o modelos de clasificación. 4) Determina la categoría preliminar. 5) Registra el resultado para la siguiente etapa.
Postcondición	El incidente queda con evidencias analizadas y datos derivados listos para su clasificación, priorización y asignación.
Excepciones	Evidencia insuficiente; archivos corruptos; error en el procesamiento automático de audio, imagen o texto.

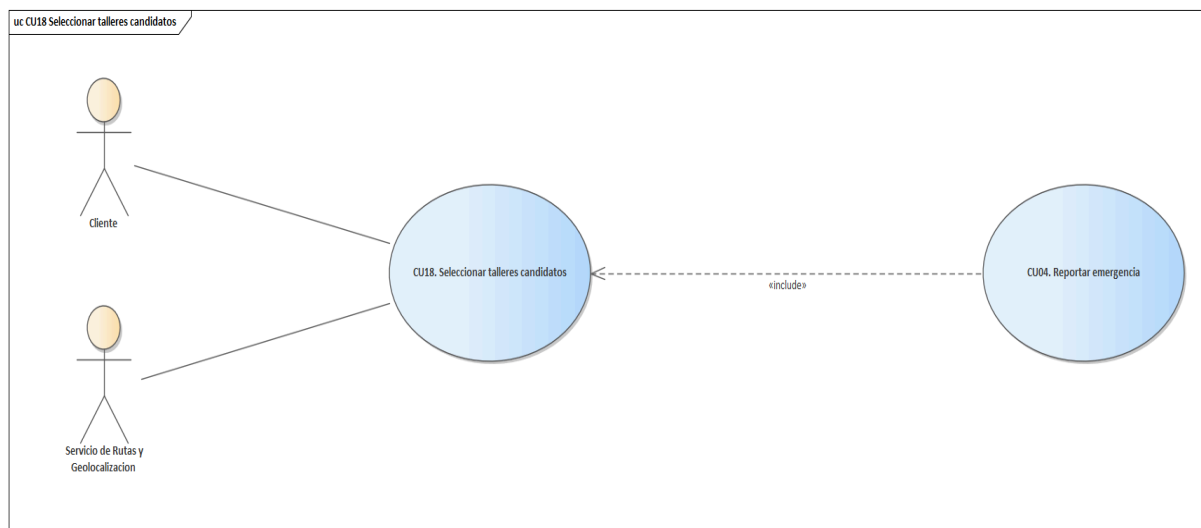
CU17 Clasificar y priorizar incidente



CAMPO	DETALLE
Nombre del caso de uso	CU17 Clasificar y priorizar incidente
Propósito	Clasificar y priorizar el incidente para orientar su tratamiento operativo y la selección de talleres candidatos. Este caso de uso se ejecuta automáticamente como parte del procesamiento posterior al reporte de emergencia.
Actores	Cliente
Actor iniciador	Cliente
Precondición	El incidente debe contar con información mínima y clasificación preliminar.
Flujo principal	1) El sistema revisa la clasificación del incidente. 2) Considera ubicación, gravedad, tipo de falla y contexto. 3) Aplica reglas de prioridad. 4) Asigna un nivel preliminar de atención.
Postcondición	El incidente queda con una categoría preliminar y un nivel de prioridad asignados para orientar su atención.
Excepciones	Información insuficiente; clasificación ambigua; error en reglas o modelos de evaluación.

Ciclo #3

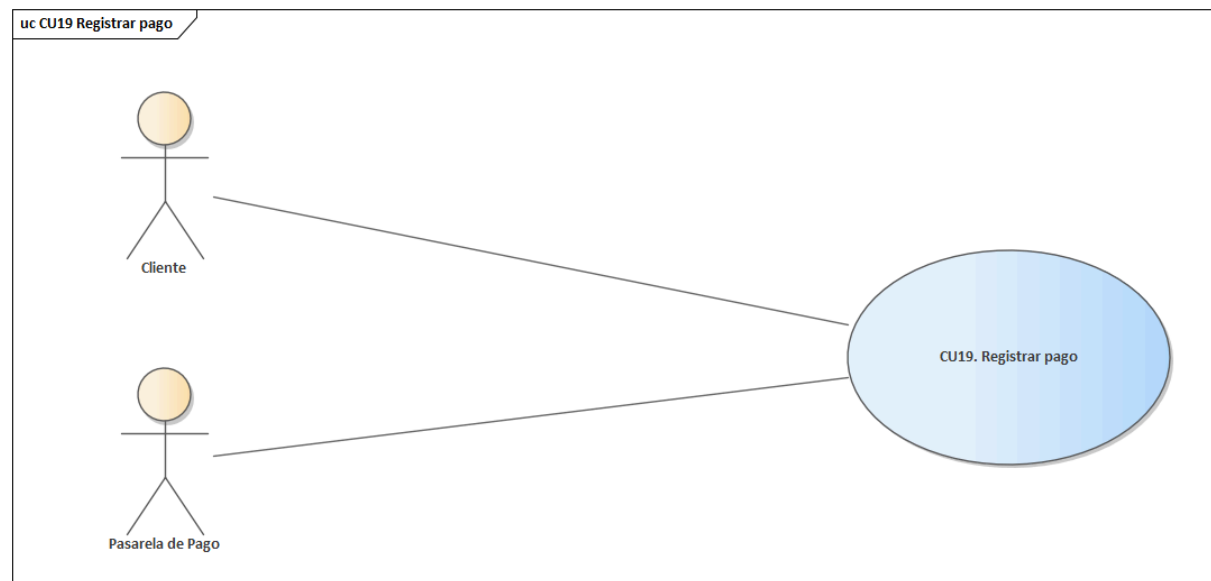
CU18 Seleccionar talleres candidatos



CAMPO	DETALLE
Nombre del caso de uso	CU18 Seleccionar talleres candidatos
Propósito	Seleccionar los talleres más adecuados para atender un incidente según criterios operativos. Este caso de uso se ejecuta automáticamente como parte del procesamiento posterior al reporte de emergencia.
Actores	Cliente; Servicio de Rutas y Geolocalización
Actor iniciador	Cliente
Precondición	El incidente debe estar clasificado y priorizado.
Flujo principal	1) El sistema toma la clasificación, prioridad y ubicación del incidente. 2) Consulta talleres compatibles. 3) Evalúa cercanía, especialidad y disponibilidad. 4) Ordena los resultados. 5) Registra la lista de candidatos.
Postcondición	Se obtiene una lista ordenada de talleres candidatos para su eventual

	notificación o atención.
Excepciones	No existen talleres compatibles; capacidad saturada; ubicación sin cobertura suficiente.

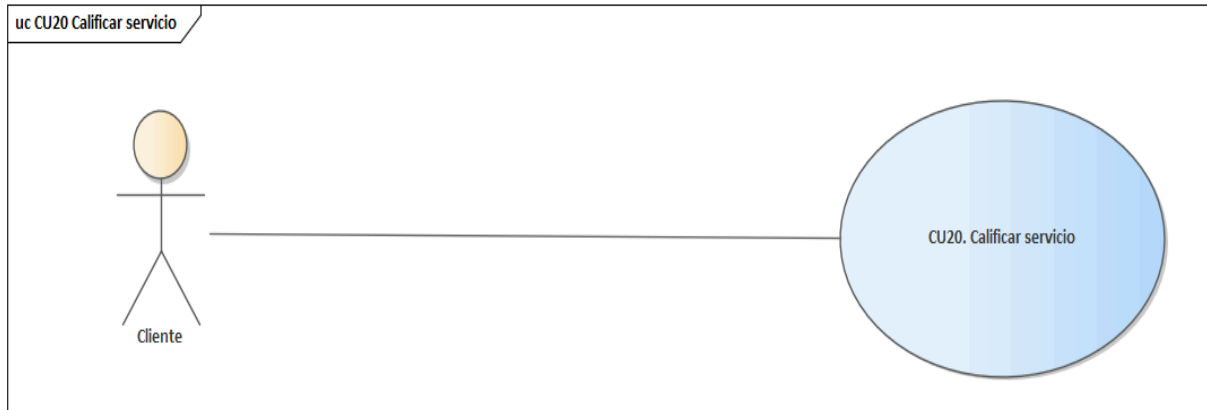
CU19 Registrar pago



CAMPO	DETALLE
Nombre del caso de uso	CU19 Registrar pago
Propósito	Registrar el pago realizado por el cliente desde la aplicación móvil al finalizar la atención.
Actores	Cliente; Pasarela de Pago
Actor iniciador	Cliente
Precondición	La atención debe estar finalizada o lista para cierre y el cliente debe tener acceso al proceso de pago desde la aplicación.
Flujo principal	1) 1) El sistema muestra el costo final y la orden de pago. 2) El cliente selecciona el método de pago desde la aplicación móvil. 3) Se procesa o valida la transacción mediante la pasarela de pago. 4) El sistema actualiza

	el estado financiero.
Postcondición	El pago queda registrado y asociado al incidente.
Excepciones	Error de validación; pago no confirmado.

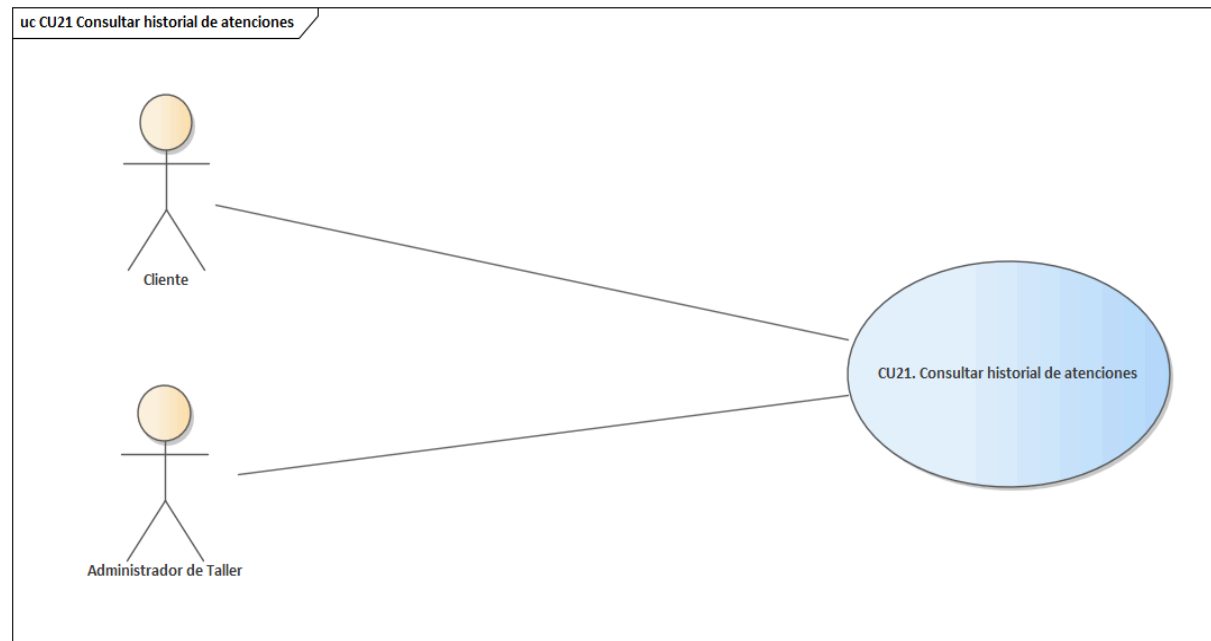
CU20 Calificar servicio



CAMPO	DETALLE
Nombre del caso de uso	CU20 Calificar servicio
Propósito	Permitir que el cliente evalúe la calidad del servicio recibido una vez concluida la atención.
Actores	Cliente
Actor iniciador	Cliente
Precondición	El incidente debe encontrarse finalizado y el cliente debe estar autenticado.
Flujo principal	1) El cliente accede al detalle del servicio finalizado. 2) Selecciona una puntuación. 3) Puede añadir comentario opcional. 4) Confirma la calificación. 5) El sistema guarda la evaluación y la asocia al historial.
Postcondición	La evaluación del cliente queda registrada y vinculada al incidente o servicio.
Excepciones	Atención aún no finalizada; intento

	duplicado de calificación; error al guardar comentario.
--	---

CU21 Consultar historial de atenciones



CAMPO	DETALLE
Nombre del caso de uso	CU21 Consultar historial de atenciones
Propósito	Permitir la consulta del historial de incidentes y atenciones previamente registradas.
Actores	Cliente; Administrador de Taller
Actor iniciador	Cliente
Precondición	El usuario debe encontrarse autenticado y contar con registros históricos.
Flujo principal	1) El usuario accede al módulo de historial. 2) El sistema recupera incidentes cerrados o atendidos. 3) Ordena la información por fecha o estado. 4) Muestra el detalle resumido de cada atención.
Postcondición	El usuario visualiza el historial

	consolidado de atenciones o servicios previos.
Excepciones	No existen atenciones previas; error de consulta histórica; datos parcialmente sincronizados.

CU22 Gestionar métricas y reportes

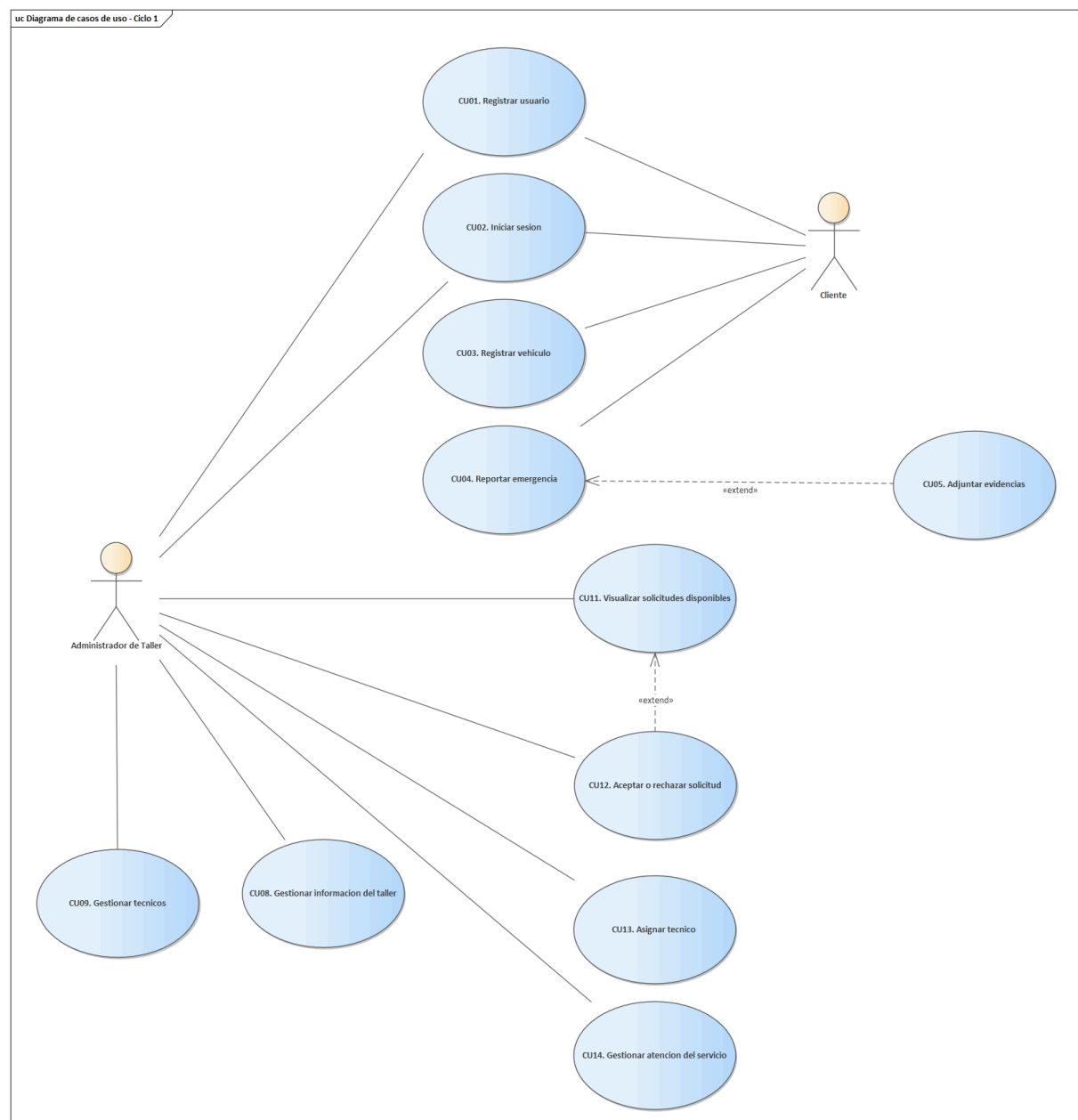


CAMPO	DETALLE
Nombre del caso de uso	CU22 Gestionar métricas y reportes
Propósito	Permitir la consulta de indicadores operativos, reportes históricos y métricas clave del sistema.
Actores	Administrador de Taller
Actor iniciador	Administrador de Taller
Precondición	Debe existir información registrada en el sistema y el usuario debe tener permisos de consulta.
Flujo principal	1) El usuario accede al módulo de métricas. 2) Selecciona período o filtro. 3) El sistema consulta ventas, incidentes y pagos. 4) Muestra indicadores, resúmenes y tendencias. 5) El usuario puede exportar o profundizar el análisis.
Postcondición	Los indicadores y reportes quedan

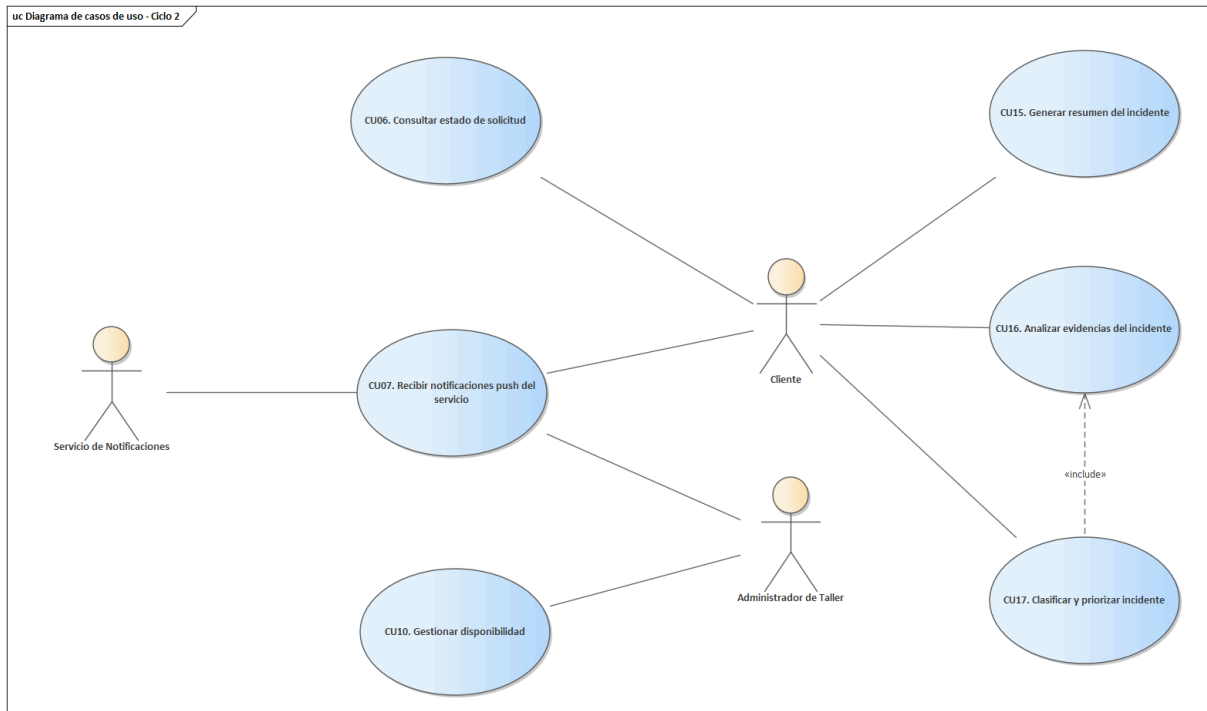
	disponibles para consulta o exportación.
Excepciones	Visualización incompleta de indicadores; permisos insuficientes para acceder a reportes; ausencia temporal de datos.

3.1.5 Estructurar Modelo de Casos de Uso

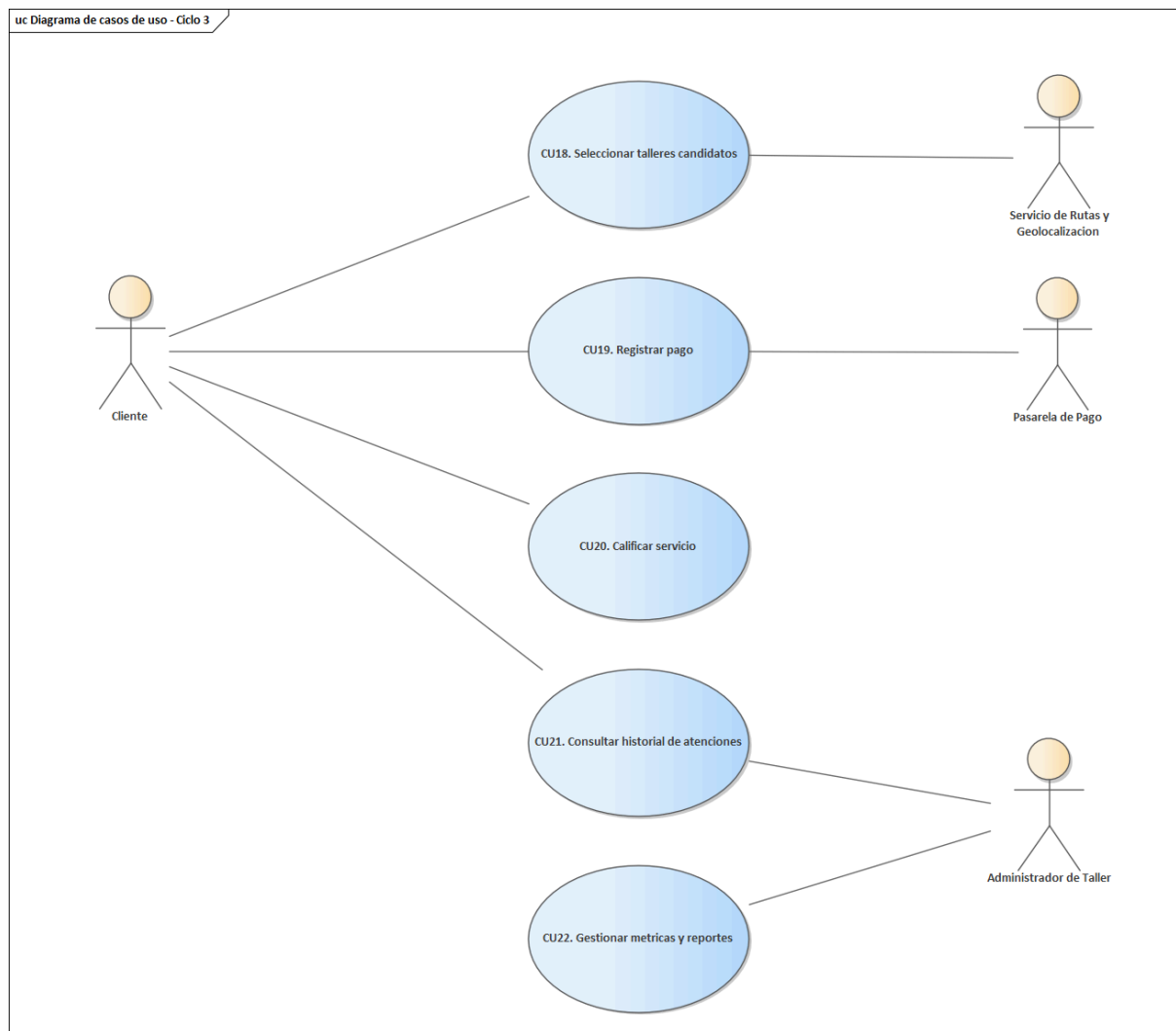
Ciclo #1



Ciclo #2

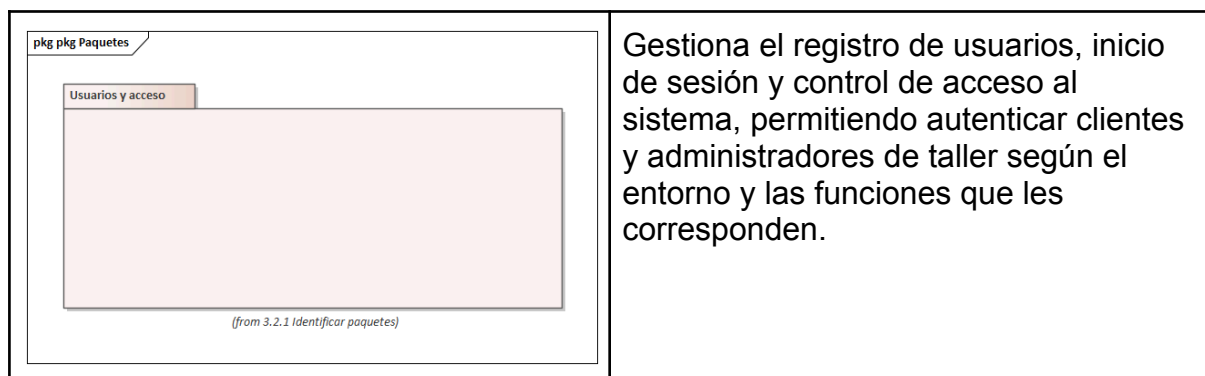


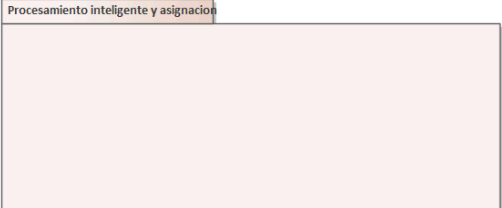
Ciclo #3

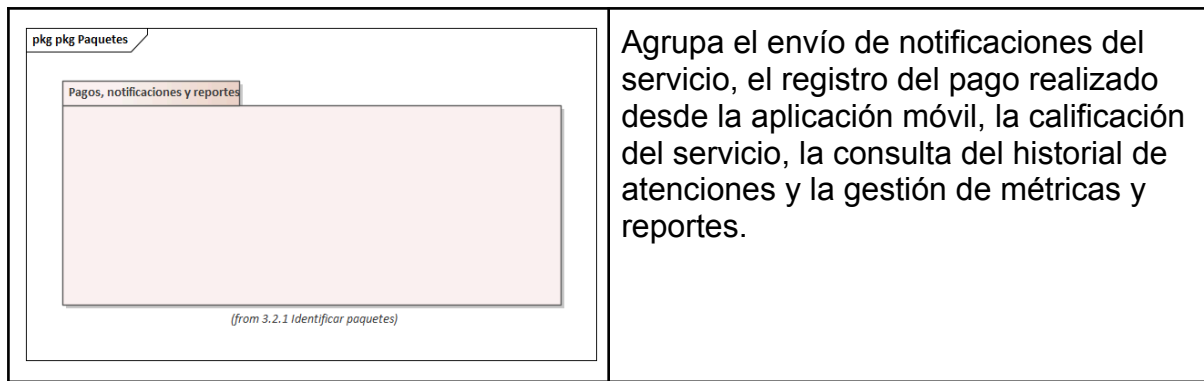


3.2 FT: ANÁLISIS

3.2.1 Identificar paquetes

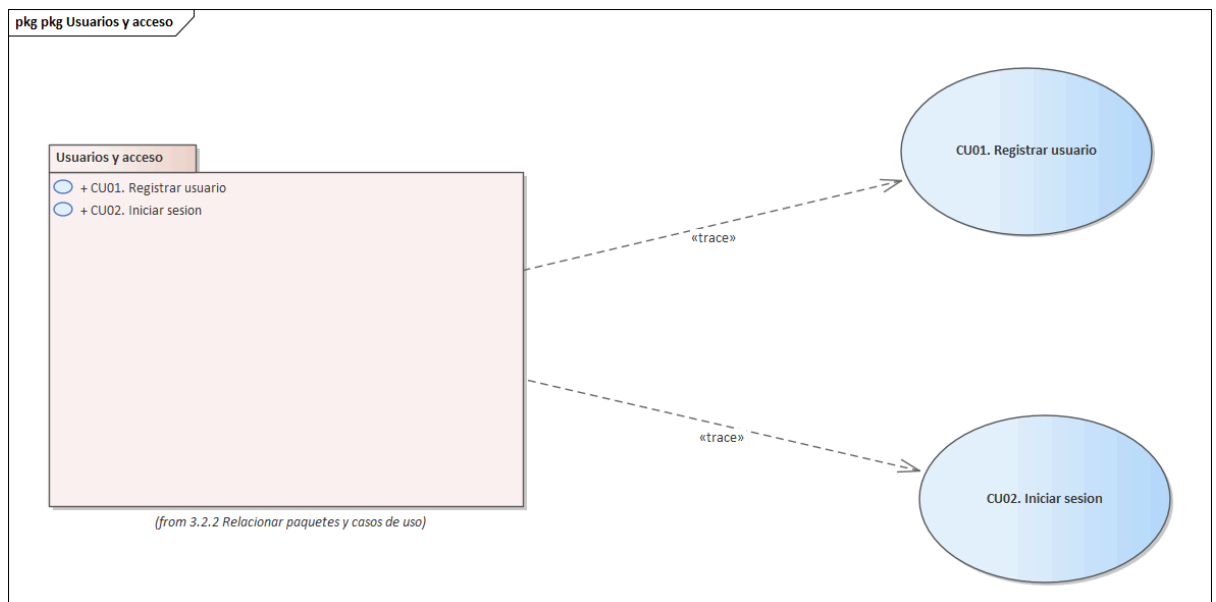


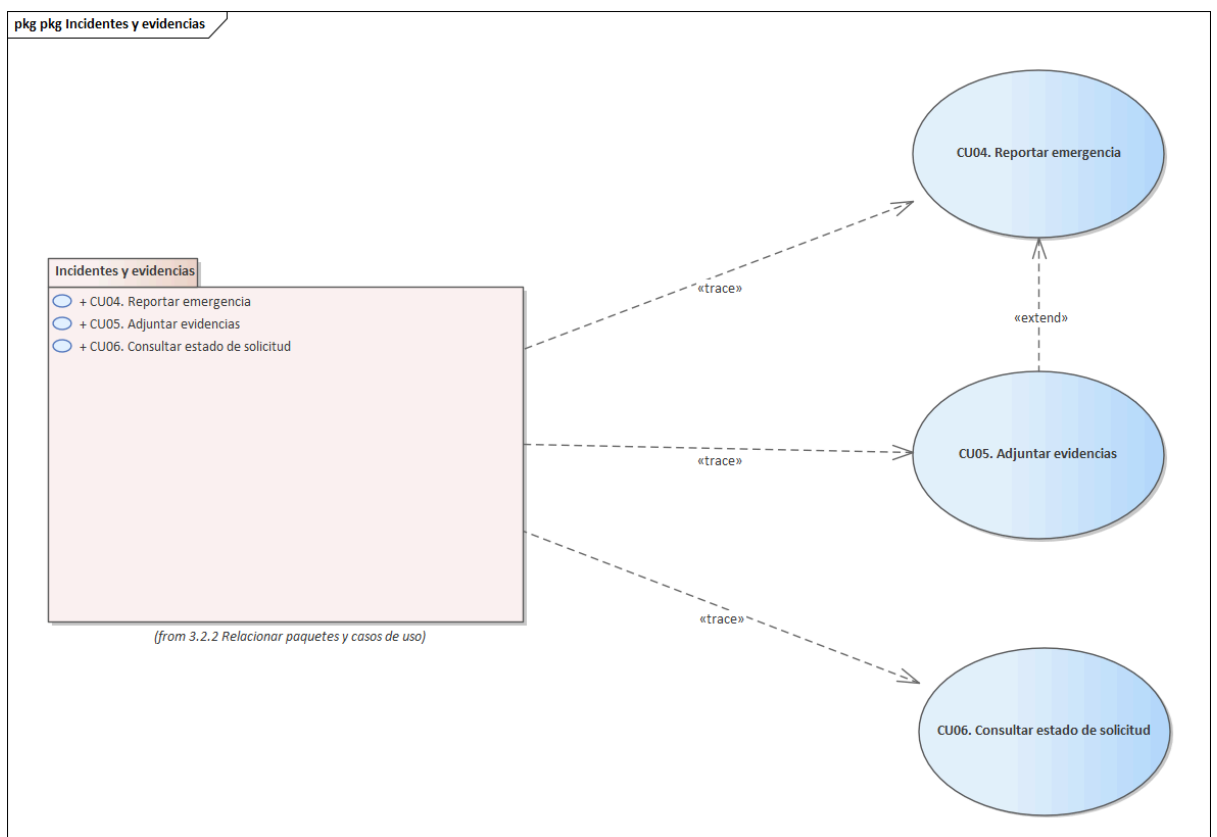
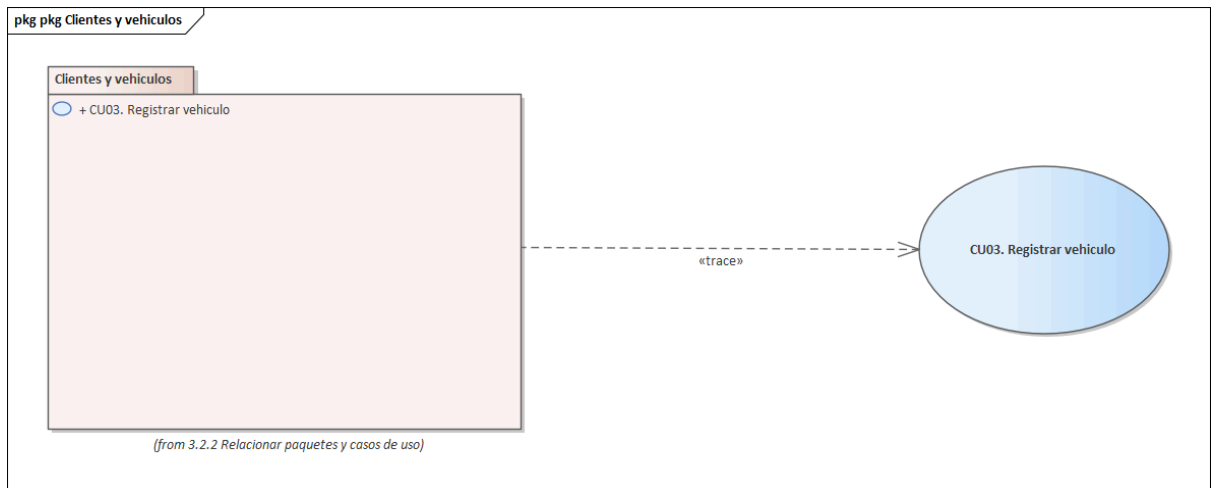
pkg pkg Paquetes Cientes y vehiculos  (from 3.2.1 Identificar paquetes)	Administra la información del cliente y el registro de sus vehículos, así como los datos necesarios para operar desde la aplicación móvil y asociar vehículos a futuras solicitudes de emergencia.
pkg pkg Paquetes Incidentes y evidencias  (from 3.2.1 Identificar paquetes)	Concentra el registro de emergencias, el almacenamiento de texto, imágenes y audios, además de la consulta del estado de la solicitud y la trazabilidad básica del incidente durante su ciclo de atención.
pkg pkg Paquetes Talleres y atencion del servicio  (from 3.2.1 Identificar paquetes)	Administra la información operativa del taller, la gestión de técnicos y disponibilidad, así como la visualización, aceptación, asignación y seguimiento de las solicitudes de servicio.
pkg pkg Paquetes Procesamiento inteligente y asignacion  (from 3.2.1 Identificar paquetes)	Integra el análisis inteligente del incidente, la generación de resumen, la clasificación preliminar, la priorización del caso y la construcción de la lista de talleres candidatos para su atención.

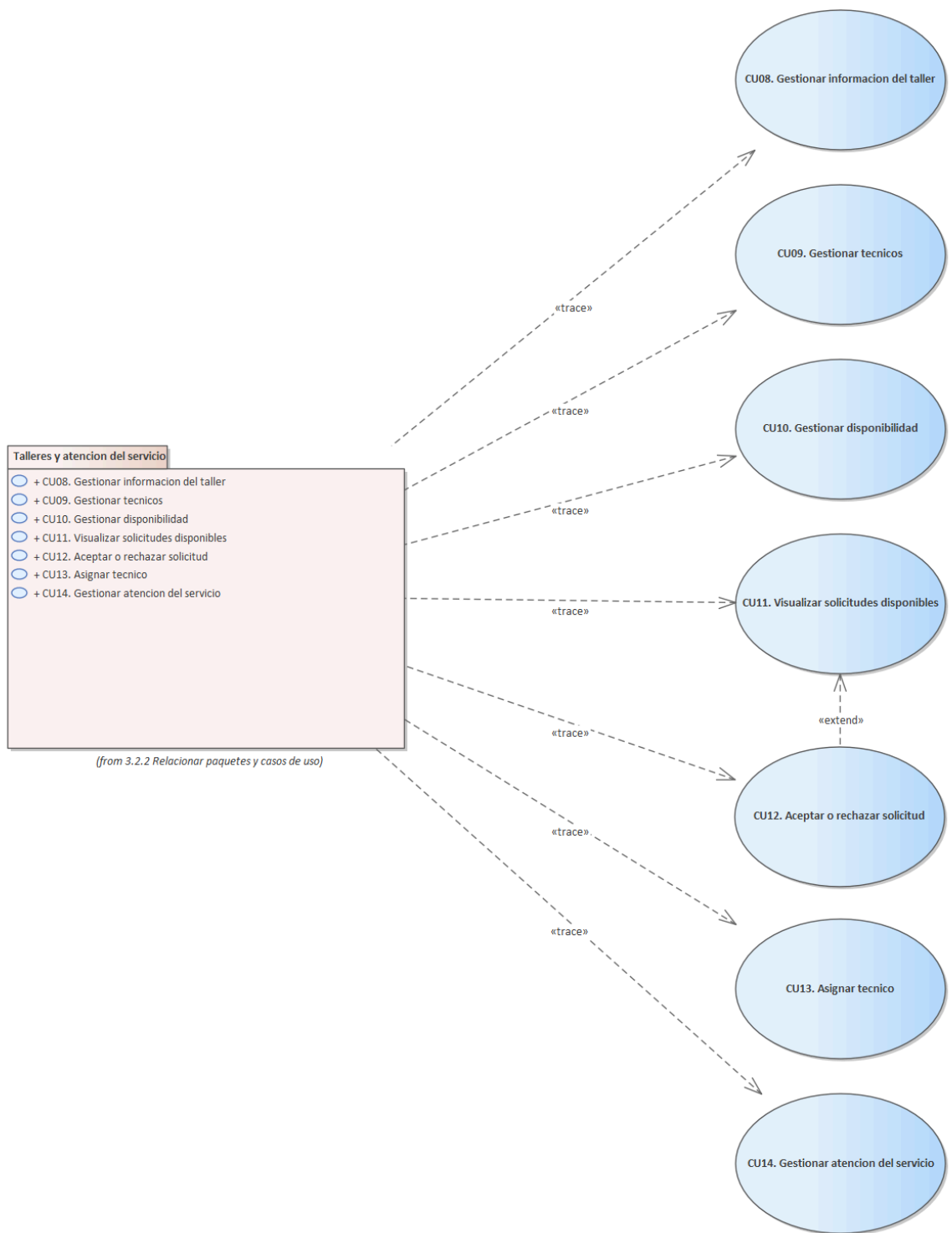


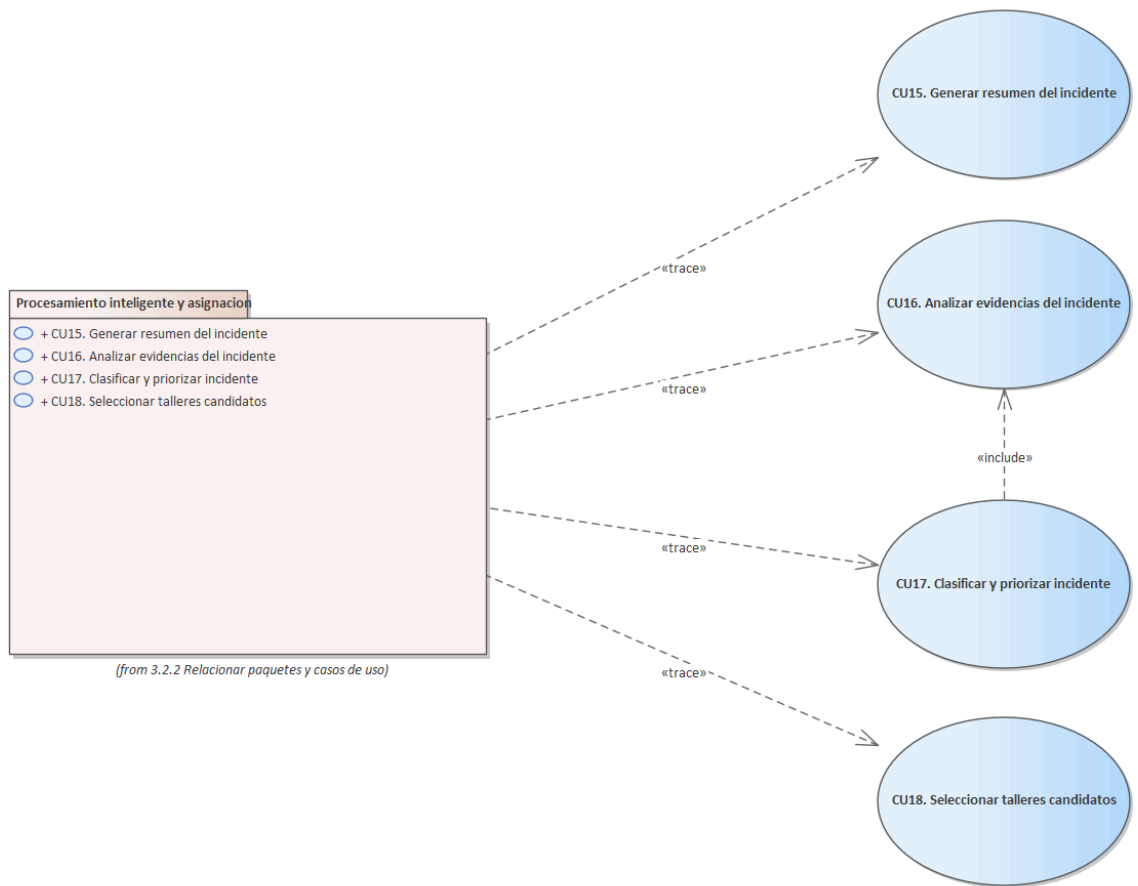
3.2.2 Relacionar paquetes y casos de uso

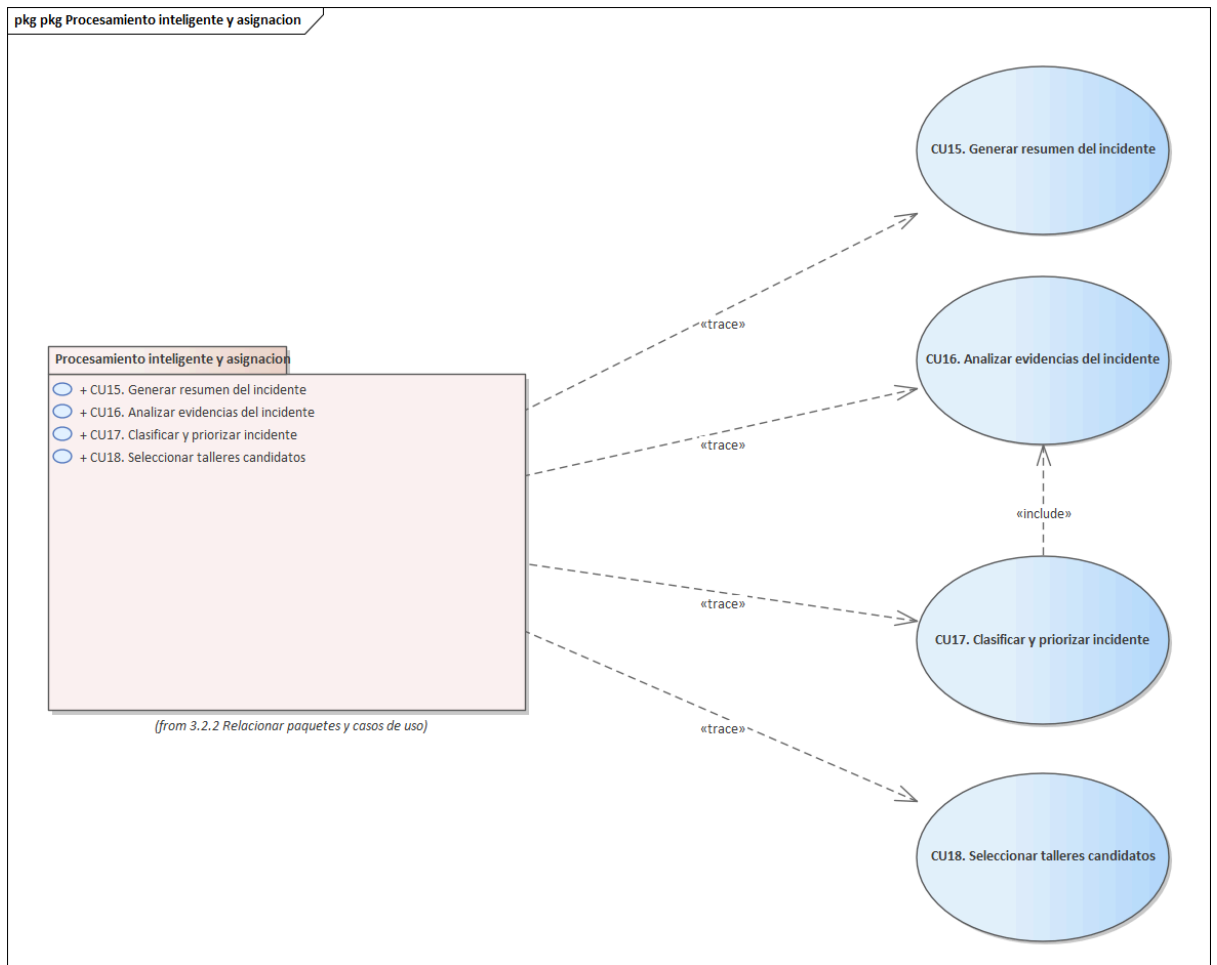
La relación entre paquetes y casos de uso puede describirse de la siguiente manera:



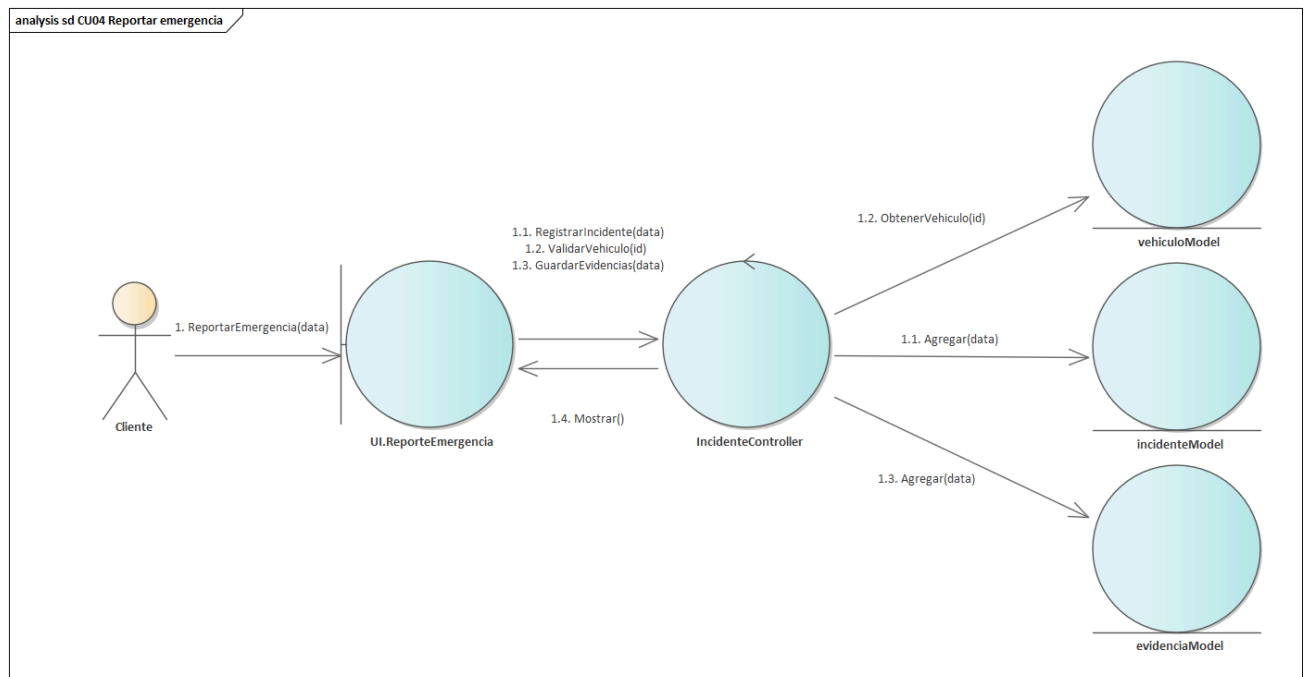




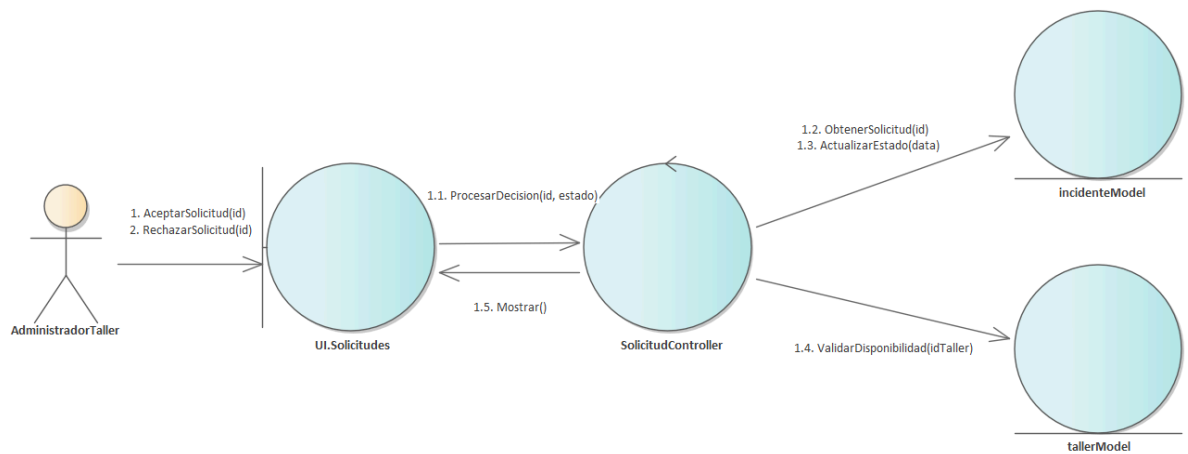




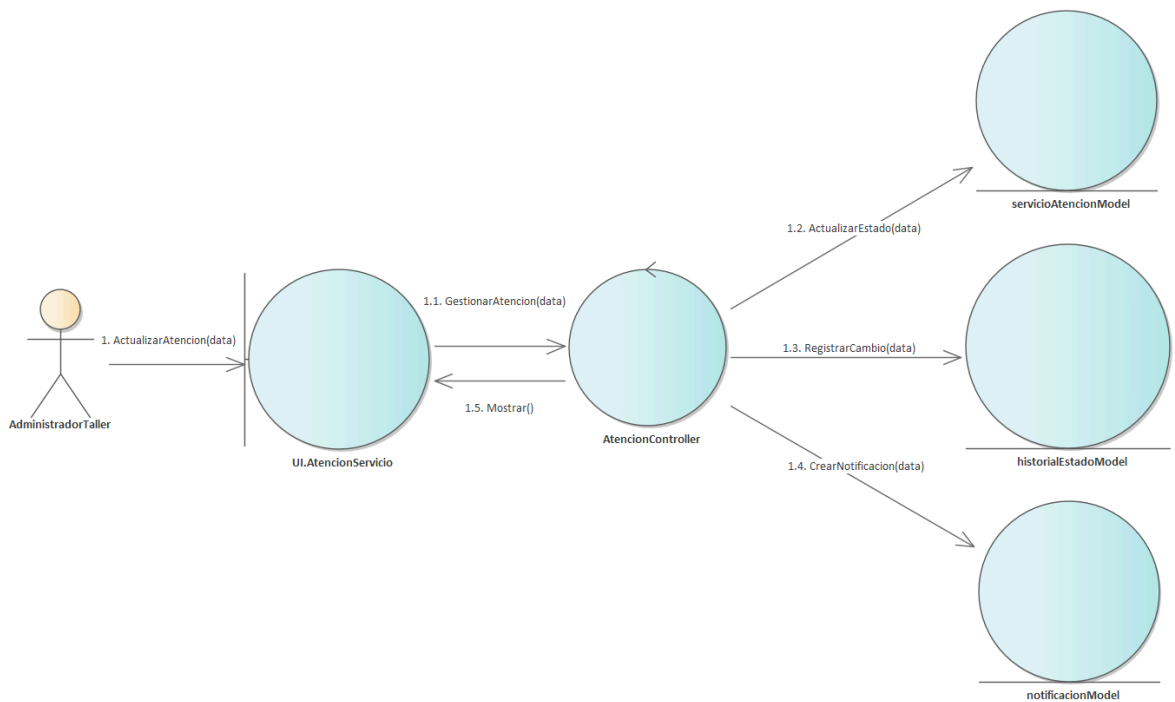
3.2.3 Diagramas de Comunicación/Colaboración



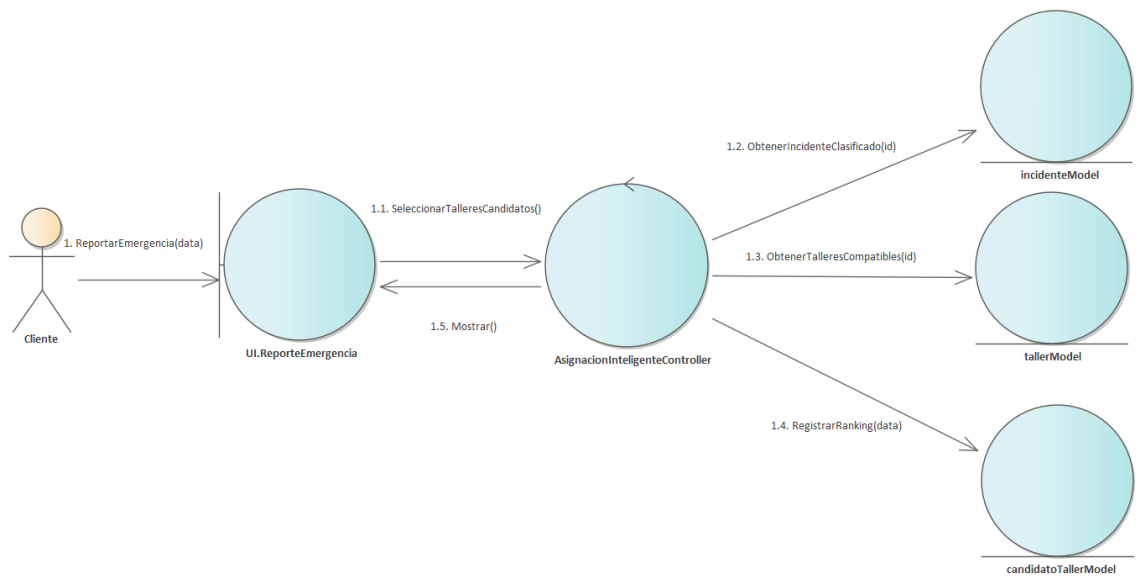
analysis sd CU12 Aceptar o rechazar solicitud



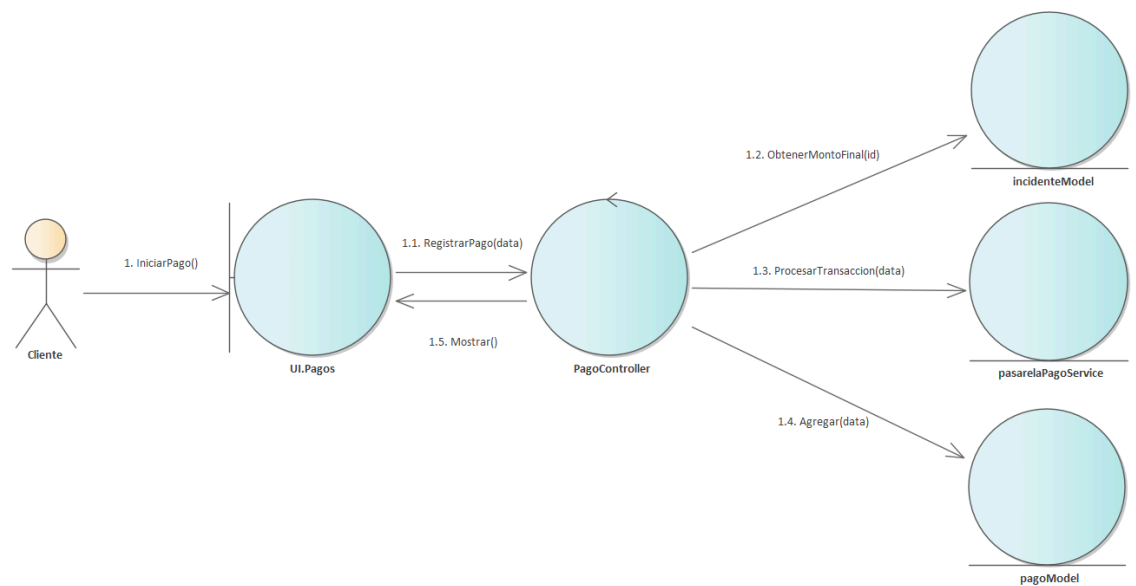
analysis sd CU14 Gestionar atencion del servicio



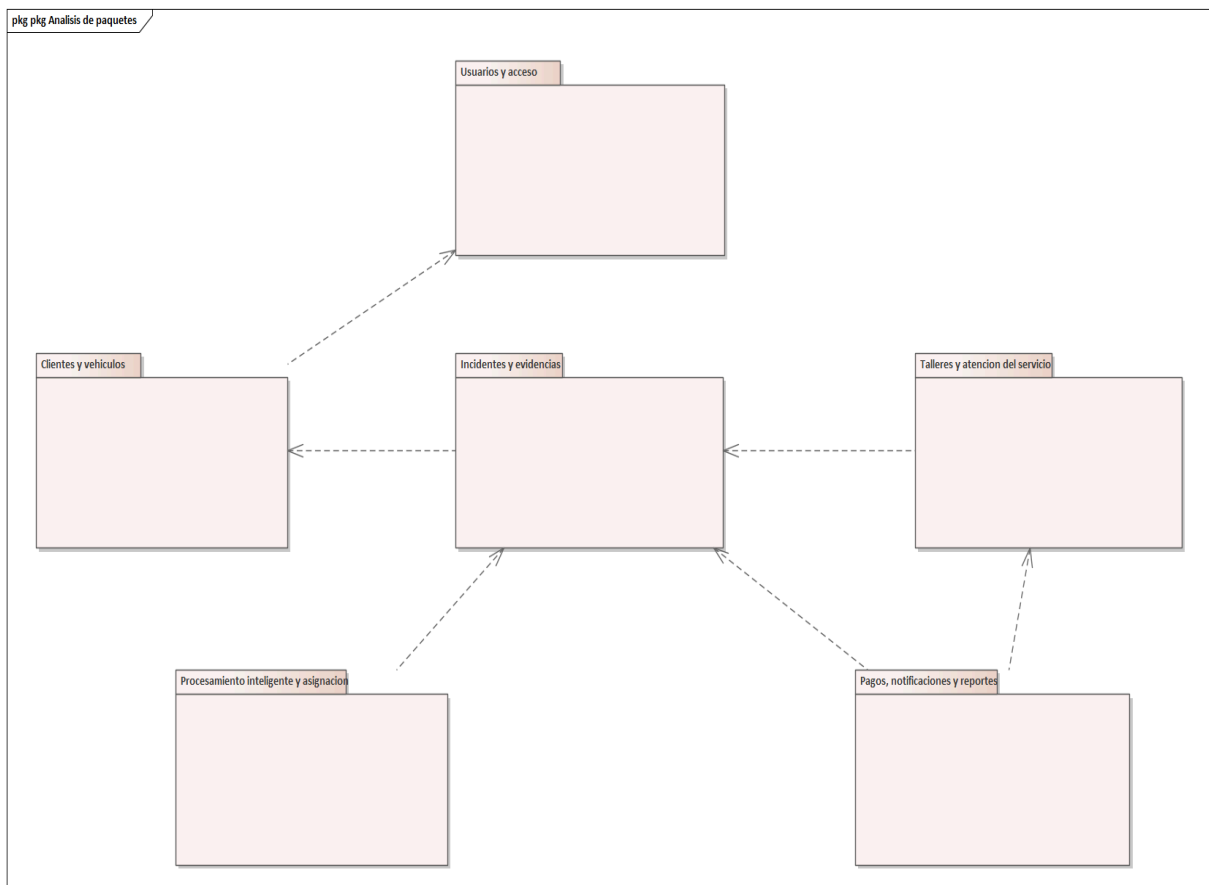
analysis sd CU18 Seleccionar talleres candidatos



analysis sd CU19 Registrar pago



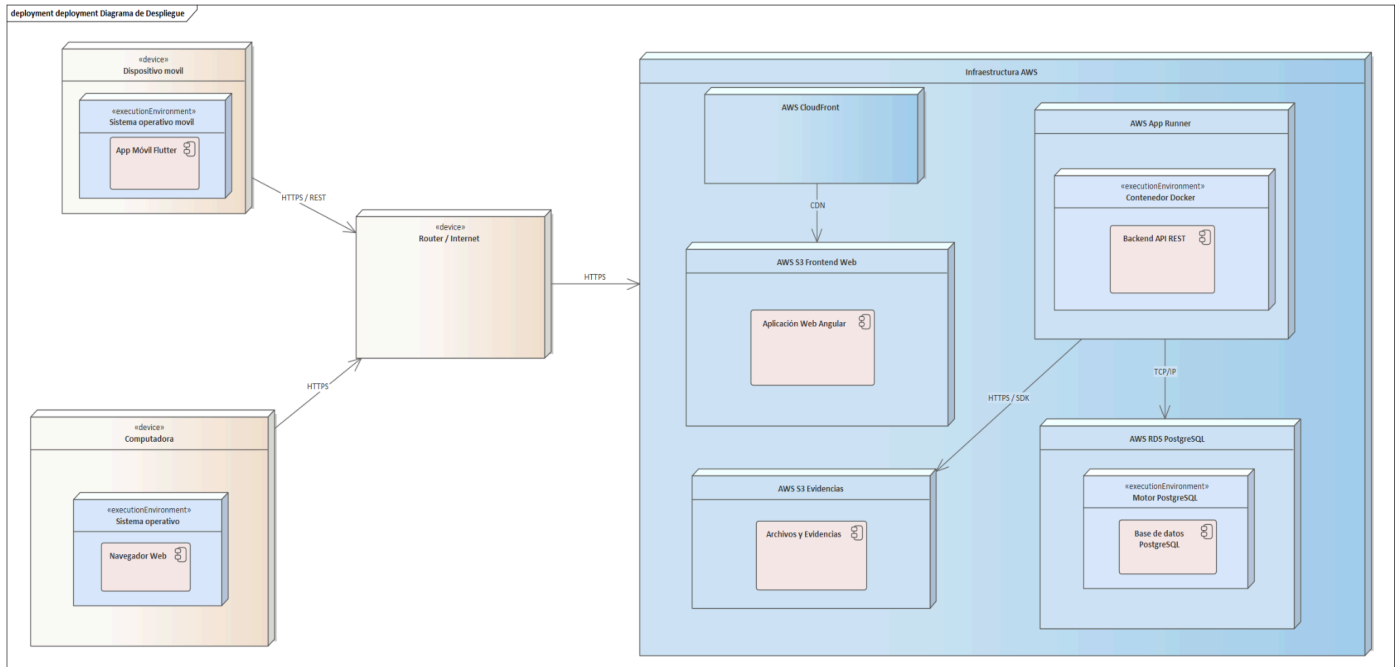
3.2.4 Análisis de Paquetes



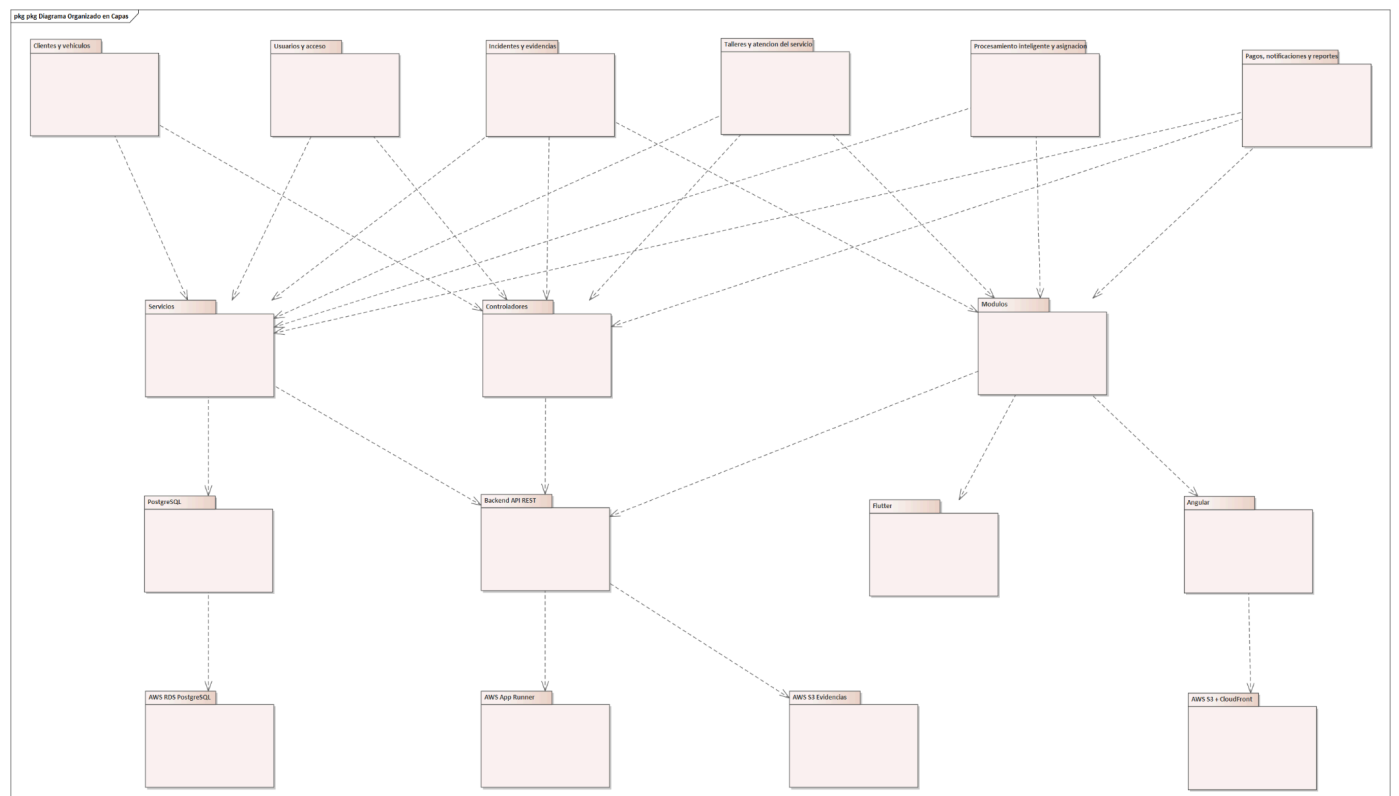
3.3 FT: DISEÑO

3.3.1 Diseño de la arquitectura

3.3.1.1 Diseño físico (Diagrama de Despliegue)



3.3.1.2 Diseño logico (Diagrama Organizado en Capas)

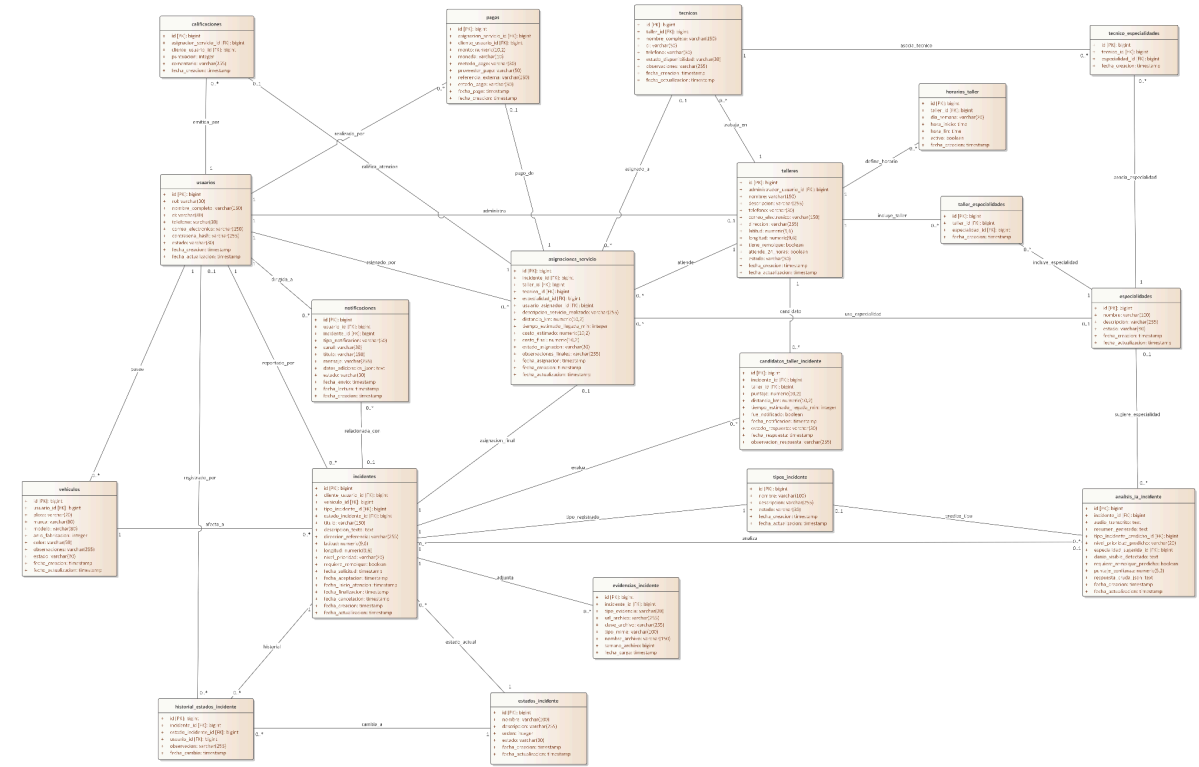


3.3.2 Diseño de Datos

3.3.2.1 Diseño de Datos Logicos

3.3.2.1.1 Diagrama de Clases

UML Diagrama de Clases Base de Datos



3.3.2.1.2 Mapeo

USUARIOS	
PK	id
FK	—
Atributos	rol nombre_completo ci telefono correo_electronico contrasena_hash estado fecha_creacion fecha_actualizacion

TALLERES

PK	id
FK	administrador_usuario_id → usuarios.id
Atributos	nombre descripcion telefono correo_electronico direccion latitud longitud tiene_remolque atiende_24_horas estado fecha_creacion fecha_actualizacion

HORARIOS_TALLER	
PK	id
FK	taller_id → talleres.id
Atributos	dia_semana hora_inicio hora_fin activo fecha_creacion

VEHICULOS	
PK	id
FK	usuario_id → usuarios.id
Atributos	placa marca modelo anio_fabricacion color observaciones estado fecha_creacion fecha_actualizacion

ESPECIALIDADES	
PK	id
FK	—
Atributos	nombre descripcion estado fecha_creacion fecha_actualizacion

TALLER_ESPECIALIDADES	
PK	id
FK	taller_id → talleres.id especialidad_id → especialidades.id
Atributos	fecha_creacion

TECNICOS	
PK	id
FK	taller_id → talleres.id
Atributos	nombre_completo ci telefono estado_disponibilidad observaciones fecha_creacion fecha_actualizacion

TECNICO_ESPECIALIDADES	
PK	id
FK	tecnico_id → tecnicos.id especialidad_id → especialidades.id

Atributos	fecha_creacion
------------------	----------------

TIPOS_INCIDENTE	
PK	id
FK	—
Atributos	nombre descripcion estado fecha_creacion fecha_actualizacion

ESTADOS_INCIDENTE	
PK	id
FK	—
Atributos	nombre descripcion orden estado fecha_creacion fecha_actualizacion

INCIDENTES	
PK	id
FK	cliente_usuario_id → usuarios.id vehiculo_id → vehiculos.id tipo_incidente_id → tipos_incidente.id estado_incidente_id → estados_incidente.id

Atributos	titulo descripcion_texto direccion_referencia latitud longitud nivel_prioridad requiere_remolque fecha_solicitud fecha_aceptacion fecha_inicio_atencion fecha_finalizacion fecha_cancelacion fecha_creacion fecha_actualizacion
------------------	--

EVIDENCIAS_INCIDENTE	
PK	id
FK	incidente_id → incidentes.id
Atributos	tipo_evidencia url_archivo clave_archivo tipo_mime nombre_archivo tamano_archivo fecha_carga

ANALISIS_IA_INCIDENTE	
PK	id
FK	incidente_id → incidentes.id tipo_incidente_predicho_id → tipos_incidente.id especialidad_sugerida_id → especialidades.id
Atributos	audio_transcrito resumen_generado nivel_prioridad_predicho danio_visible_detectado requiere_remolque_predicho puntaje_confianza respuesta_cruda_json fecha_creacion fecha_actualizacion

HISTORIAL_ESTADOS_INCIDENTE	
PK	id
FK	incidente_id → incidentes.id estado_incidente_id → estados_incidente.id usuario_id → usuarios.id
Atributos	observacion fecha_cambio

CANDIDATOS_TALLER_INCIDENTE	
PK	id
FK	incidente_id → incidentes.id taller_id → talleres.id
Atributos	puntaje distancia_km tiempo_estimado_llegada_min fue_notificado fecha_notificacion estado_respuesta fecha_respuesta observacion_respuesta

ASIGNACIONES_SERVICIO	
PK	id
FK	incidente_id → incidentes.id taller_id → talleres.id tecnico_id → tecnicos.id especialidad_id → especialidades.id usuario_asignador_id → usuarios.id

Atributos	descripcion_servicio_realizado distancia_km tiempo_estimado_llegada_min costo_estimado costo_final estado_asignacion observaciones_finales fecha_asignacion fecha_creacion fecha_actualizacion
------------------	---

PAGOS	
PK	id
FK	asignacion_servicio_id → asignaciones_servicio.id cliente_usuario_id → usuarios.id
Atributos	monto moneda metodo_pago proveedor_pago referencia_externa estado_pago fecha_pago fecha_creacion

CALIFICACIONES	
PK	id
FK	asignacion_servicio_id → asignaciones_servicio.id cliente_usuario_id → usuarios.id
Atributos	puntuacion comentario fecha_creacion

NOTIFICACIONES	
PK	id

FK	usuario_id → usuarios.id incidente_id → incidentes.id
Atributos	tipo_notificacion canal titulo mensaje datos_adicionales_json estado fecha_envio fecha_lectura fecha_creacion

3.3.2.1.3 Normalización

Las clases ya se encuentran completamente normalizadas.

3.3.2.2 Diseño de Datos Físicos

3.3.2.1.1 Script

```
create table if not exists usuarios (
  id bigint generated always as identity primary key,
  rol varchar(30) not null,
  nombre_completo varchar(150) not null,
  ci varchar(30),
  telefono varchar(30),
  correo_electronico varchar(150) not null,
  contraseña_hash varchar(255) not null,
  estado varchar(30) not null default 'ACTIVO',
  fecha_creacion timestamp not null default now(),
  fecha_actualizacion timestamp not null default now(),

  constraint uq_usuarios_correo unique
(correo_electronico),
  constraint chk_usuarios_rol
  check (rol in ('CLIENTE', 'ADMIN_TALLER')),
  constraint chk_usuarios_estado
  check (estado in ('ACTIVO', 'INACTIVO',
'SUSPENDIDO'))
);

create table if not exists talleres (
  id bigint generated always as identity primary key,
  administrador_usuario_id bigint not null,
  nombre varchar(150) not null,
  descripcion varchar(255),
  telefono varchar(30),
  correo_electronico varchar(150),
  direccion varchar(255),
  latitud numeric(9,6),
```

```

    longitud numeric(9,6),
    tiene_remolque boolean not null default false,
    atiende_24_horas boolean not null default false,
    estado varchar(30) not null default 'ACTIVO',
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint uq_talleres_admin unique
(administrador_usuario_id),
    constraint fk_talleres_admin
        foreign key (administrador_usuario_id)
        references usuarios(id)
        on update cascade
        on delete restrict,
    constraint chk_talleres_estado
        check (estado in ('ACTIVO', 'INACTIVO'))
);

create table if not exists horarios_taller (
    id bigint generated always as identity primary key,
    taller_id bigint not null,
    dia_semana varchar(20) not null,
    hora_inicio time not null,
    hora_fin time not null,
    activo boolean not null default true,
    fecha_creacion timestamp not null default now(),

    constraint fk_horarios_taller_taller
        foreign key (taller_id)
        references talleres(id)
        on update cascade
        on delete cascade,
    constraint uq_horarios_taller unique (taller_id,
dia_semana, hora_inicio, hora_fin),
    constraint chk_horarios_dia_semana
        check (dia_semana in (
            'LUNES', 'MARTES', 'MIERCOLES', 'JUEVES',
            'VIERNES', 'SABADO', 'DOMINGO'
        )),
    constraint chk_horarios_rango
        check (hora_fin > hora_inicio)
);

create table if not exists vehiculos (
    id bigint generated always as identity primary key,
    usuario_id bigint not null,
    placa varchar(20) not null,
    marca varchar(80) not null,
    modelo varchar(80) not null,
    anio_fabricacion integer,
    color varchar(50),

```

```

    observaciones varchar(255),
    estado varchar(30) not null default 'ACTIVO',
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint uq_vehiculos_placa unique (placa),
    constraint fk_vehiculos_usuario
        foreign key (usuario_id)
        references usuarios(id)
        on update cascade
        on delete restrict,
    constraint chk_vehiculos_estado
        check (estado in ('ACTIVO', 'INACTIVO')),
    constraint chk_vehiculos_anio
        check (anio_fabricacion is null or
anio_fabricacion between 1950 and 2100)
);

create table if not exists especialidades (
    id bigint generated always as identity primary key,
    nombre varchar(100) not null,
    descripcion varchar(255),
    estado varchar(30) not null default 'ACTIVO',
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint uq_especialidades_nombre unique (nombre),
    constraint chk_especialidades_estado
        check (estado in ('ACTIVO', 'INACTIVO'))
);

create table if not exists taller_especialidades (
    id bigint generated always as identity primary key,
    taller_id bigint not null,
    especialidad_id bigint not null,
    fecha_creacion timestamp not null default now(),

    constraint uq_taller_especialidad unique (taller_id,
especialidad_id),
    constraint fk_taller_especialidades_taller
        foreign key (taller_id)
        references talleres(id)
        on update cascade
        on delete cascade,
    constraint fk_taller_especialidades_especialidad
        foreign key (especialidad_id)
        references especialidades(id)
        on update cascade
        on delete restrict
);

```



```

create table if not exists tecnicos (
    id bigint generated always as identity primary key,
    taller_id bigint not null,
    nombre_completo varchar(150) not null,
    ci varchar(30),
    telefono varchar(30),
    estado_disponibilidad varchar(30) not null default
'DISPONIBLE',
    observaciones varchar(255),
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint fk_tecnicos_taller
        foreign key (taller_id)
        references talleres(id)
        on update cascade
        on delete cascade,
    constraint chk_tecnicos_estado_disponibilidad
        check (estado_disponibilidad in ('DISPONIBLE',
'OCUPADO', 'INACTIVO'))
);

create table if not exists tecnico_especialidades (
    id bigint generated always as identity primary key,
    tecnico_id bigint not null,
    especialidad_id bigint not null,
    fecha_creacion timestamp not null default now(),

    constraint uq_tecnico_especialidad unique
(tecnico_id, especialidad_id),
    constraint fk_tecnico_especialidades_tecnico
        foreign key (tecnico_id)
        references tecnicos(id)
        on update cascade
        on delete cascade,
    constraint fk_tecnico_especialidades_especialidad
        foreign key (especialidad_id)
        references especialidades(id)
        on update cascade
        on delete restrict
);

create table if not exists tipos_incidente (
    id bigint generated always as identity primary key,
    nombre varchar(100) not null,
    descripcion varchar(255),
    estado varchar(30) not null default 'ACTIVO',
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint uq_tipos_incidente_nombre unique (nombre),

```

```

        constraint chk_tipos_incidente_estado
            check (estado in ('ACTIVO', 'INACTIVO'))
    );

create table if not exists estados_incidente (
    id bigint generated always as identity primary key,
    nombre varchar(100) not null,
    descripcion varchar(255),
    orden integer not null,
    estado varchar(30) not null default 'ACTIVO',
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint uq_estados_incidente_nombre unique
(nombre),
    constraint uq_estados_incidente_orden unique (orden),
    constraint chk_estados_incidente_estado
        check (estado in ('ACTIVO', 'INACTIVO')),
    constraint chk_estados_incidente_orden
        check (orden > 0)
);

create table if not exists incidentes (
    id bigint generated always as identity primary key,
    cliente_usuario_id bigint not null,
    vehiculo_id bigint not null,
    tipo_incidente_id bigint,
    estado_incidente_id bigint not null,
    titulo varchar(150) not null,
    descripcion_texto text,
    direccion_referencia varchar(255),
    latitud numeric(9,6),
    longitud numeric(9,6),
    nivel_prioridad varchar(20),
    requiere_remolque boolean not null default false,
    fecha_solicitud timestamp not null default now(),
    fecha_aceptacion timestamp,
    fecha_inicio_atencion timestamp,
    fecha_finalizacion timestamp,
    fecha_cancelacion timestamp,
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint fk_incidentes_cliente
        foreign key (cliente_usuario_id)
        references usuarios(id)
        on update cascade
        on delete restrict,
    constraint fk_incidentes_vehiculo
        foreign key (vehiculo_id)
        references vehiculos(id)
);

```

```

        on update cascade
        on delete restrict,
constraint fk_incidentes_tipo
    foreign key (tipo_incidente_id)
    references tipos_incidente(id)
    on update cascade
    on delete set null,
constraint fk_incidentes_estado
    foreign key (estado_incidente_id)
    references estados_incidente(id)
    on update cascade
    on delete restrict,
constraint chk_incidentes_prioridad
    check (
        nivel_prioridad is null
        or nivel_prioridad in ('BAJA', 'MEDIA',
'ALTA', 'CRITICA', 'INCIERTA')
    )
);

create table if not exists evidencias_incidente (
    id bigint generated always as identity primary key,
    incidente_id bigint not null,
    tipo_evidencia varchar(20) not null,
    url_archivo varchar(255),
    clave_archivo varchar(255),
    tipo_mime varchar(100),
    nombre_archivo varchar(150),
    tamaño_archivo bigint,
    fecha_carga timestamp not null default now(),

    constraint fk_evidencias_incidente
        foreign key (incidente_id)
        references incidentes(id)
        on update cascade
        on delete cascade,
    constraint chk_evidencias_tipo
        check (tipo_evidencia in ('IMAGEN', 'AUDIO',
'VIDEO', 'DOCUMENTO')),
    constraint chk_evidencias_tamaño
        check (tamaño_archivo is null or tamaño_archivo
>= 0)
);

create table if not exists analisis_ia_incidente (
    id bigint generated always as identity primary key,
    incidente_id bigint not null,
    audio_transcrito text,
    resumen_generado text,
    tipo_incidente_predicho_id bigint,
    nivel_prioridad_predicho varchar(20),

```

```

    especialidad_sugerida_id bigint,
    danio_visible_detectado text,
    requiere_remolque_predicho boolean,
    puntaje_confianza numeric(5,2),
    respuesta_cruda_json text,
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint uq_analisis_ia_incidente unique
(incidente_id),
    constraint fk_analisis_ia_incidente
        foreign key (incidente_id)
        references incidentes(id)
        on update cascade
        on delete cascade,
    constraint fk_analisis_ia_tipo_predicho
        foreign key (tipo_incidente_predicho_id)
        references tipos_incidente(id)
        on update cascade
        on delete set null,
    constraint fk_analisis_ia_especialidad
        foreign key (especialidad_sugerida_id)
        references especialidades(id)
        on update cascade
        on delete set null,
    constraint chk_analisis_ia_prioridad
        check (
            nivel_prioridad_predicho is null
            or nivel_prioridad_predicho in ('BAJA',
'MEDIA', 'ALTA', 'CRITICA', 'INCIERTA')
        ),
    constraint chk_analisis_ia_confianza
        check (puntaje_confianza is null or
puntaje_confianza between 0 and 100)
);

create table if not exists historial_estados_incidente (
    id bigint generated always as identity primary key,
    incidente_id bigint not null,
    estado_incidente_id bigint not null,
    usuario_id bigint,
    observacion varchar(255),
    fecha_cambio timestamp not null default now(),

    constraint fk_historial_incidente
        foreign key (incidente_id)
        references incidentes(id)
        on update cascade
        on delete cascade,
    constraint fk_historial_estado
        foreign key (estado_incidente_id)

```

```

        references estados_incidente(id)
        on update cascade
        on delete restrict,
    constraint fk_historial_usuario
        foreign key (usuario_id)
        references usuarios(id)
        on update cascade
        on delete set null
);

create table if not exists candidatos_taller_incidente (
    id bigint generated always as identity primary key,
    incidente_id bigint not null,
    taller_id bigint not null,
    puntaje numeric(10,2),
    distancia_km numeric(10,2),
    tiempo_estimado_llegada_min integer,
    fue_notificado boolean not null default false,
    fecha_notificacion timestamp,
    estado_respuesta varchar(30) not null default
'PENDIENTE',
    fecha_respuesta timestamp,
    observacion_respuesta varchar(255),

    constraint uq_candidato_taller_incidente unique
(incidente_id, taller_id),
    constraint fk_candidatos_incidente
        foreign key (incidente_id)
        references incidentes(id)
        on update cascade
        on delete cascade,
    constraint fk_candidatos_taller
        foreign key (taller_id)
        references talleres(id)
        on update cascade
        on delete restrict,
    constraint chk_candidatos_puntaje
        check (puntaje is null or puntaje >= 0),
    constraint chk_candidatos_distancia
        check (distancia_km is null or distancia_km >=
0),
    constraint chk_candidatos_tiempo
        check (tiempo_estimado_llegada_min is null or
tiempo_estimado_llegada_min >= 0),
    constraint chk_candidatos_estado_respuesta
        check (estado_respuesta in ('PENDIENTE',
'ACEPTADO', 'RECHAZADO', 'EXPIRADO'))
);

create table if not exists asignaciones_servicio (
    id bigint generated always as identity primary key,

```

```

    incidente_id bigint not null,
    taller_id bigint not null,
    tecnico_id bigint,
    especialidad_id bigint,
    usuario_asignador_id bigint,
    descripcion_servicio_realizado text,
    distancia_km numeric(10,2),
    tiempo_estimado_llegada_min integer,
    costo_estimado numeric(10,2),
    costo_final numeric(10,2),
    estado_asignacion varchar(30) not null default
'ASIGNADO',
    observaciones_finales text,
    fecha_asignacion timestamp not null default now(),
    fecha_creacion timestamp not null default now(),
    fecha_actualizacion timestamp not null default now(),

    constraint uq_asignaciones_incidente unique
(incidente_id),
    constraint fk_asignaciones_incidente
        foreign key (incidente_id)
        references incidentes(id)
        on update cascade
        on delete cascade,
    constraint fk_asignaciones_taller
        foreign key (taller_id)
        references talleres(id)
        on update cascade
        on delete restrict,
    constraint fk_asignaciones_tecnico
        foreign key (tecnico_id)
        references tecnicos(id)
        on update cascade
        on delete set null,
    constraint fk_asignaciones_especialidad
        foreign key (especialidad_id)
        references especialidades(id)
        on update cascade
        on delete set null,
    constraint fk_asignaciones_usuario_asignador
        foreign key (usuario_asignador_id)
        references usuarios(id)
        on update cascade
        on delete set null,
    constraint chk_asignaciones_distancia
        check (distancia_km is null or distancia_km >=
0),
    constraint chk_asignaciones_tiempo
        check (tiempo_estimado_llegada_min is null or
tiempo_estimado_llegada_min >= 0),
    constraint chk_asignaciones_costo_estimado

```

```

        check (costo_estimado is null or costo_estimado
>= 0),
    constraint chk_asignaciones_costo_final
        check (costo_final is null or costo_final >= 0),
    constraint chk_asignaciones_estado
        check (
            estado_asignacion in (
                'ASIGNADO',
                'EN_CAMINO',
                'EN_PROCESO',
                'ATENDIDO',
                'CANCELADO',
                'PENDIENTE_PAGO',
                'PAGADO'
            )
        )
);

create table if not exists pagos (
    id bigint generated always as identity primary key,
    asignacion_servicio_id bigint not null,
    cliente_usuario_id bigint not null,
    monto numeric(10,2) not null,
    moneda varchar(10) not null default 'BOB',
    metodo_pago varchar(30) not null,
    proveedor_pago varchar(50),
    referencia_externa varchar(150),
    estado_pago varchar(30) not null default 'PENDIENTE',
    fecha_pago timestamp,
    fecha_creacion timestamp not null default now(),

    constraint uq_pagos_asignacion unique
(asignacion_servicio_id),
    constraint fk_pagos_asignacion
        foreign key (asignacion_servicio_id)
        references asignaciones_servicio(id)
        on update cascade
        on delete cascade,
    constraint fk_pagos_cliente
        foreign key (cliente_usuario_id)
        references usuarios(id)
        on update cascade
        on delete restrict,
    constraint chk_pagos_monto
        check (monto >= 0),
    constraint chk_pagos_metodo
        check (metodo_pago in ('QR', 'EFECTIVO',
'TRANSFERENCIA')),
    constraint chk_pagos_estado
        check (estado_pago in ('PENDIENTE', 'PAGADO',
'FALLIDO', 'CANCELADO'))
);

```

```

);

create table if not exists calificaciones (
    id bigint generated always as identity primary key,
    asignacion_servicio_id bigint not null,
    cliente_usuario_id bigint not null,
    puntuacion integer not null,
    comentario varchar(255),
    fecha_creacion timestamp not null default now(),

    constraint uq_calificaciones_asignacion unique
(asignacion_servicio_id),
    constraint fk_calificaciones_asignacion
        foreign key (asignacion_servicio_id)
        references asignaciones_servicio(id)
        on update cascade
        on delete cascade,
    constraint fk_calificaciones_cliente
        foreign key (cliente_usuario_id)
        references usuarios(id)
        on update cascade
        on delete restrict,
    constraint chk_calificaciones_puntuacion
        check (puntuacion between 1 and 5)
);

create table if not exists notificaciones (
    id bigint generated always as identity primary key,
    usuario_id bigint,
    incidente_id bigint,
    tipo_notificacion varchar(50) not null,
    canal varchar(30) not null,
    titulo varchar(150) not null,
    mensaje varchar(255) not null,
    datos_adicionales_json text,
    estado varchar(30) not null default 'PENDIENTE',
    fecha_envio timestamp,
    fecha_lectura timestamp,
    fecha_creacion timestamp not null default now(),

    constraint fk_notificaciones_usuario
        foreign key (usuario_id)
        references usuarios(id)
        on update cascade
        on delete set null,
    constraint fk_notificaciones_incidente
        foreign key (incidente_id)
        references incidentes(id)
        on update cascade
        on delete set null,
    constraint chk_notificaciones_canal

```



```

        check (canal in ('PUSH', 'EMAIL', 'SMS',
'IN_APP')),
        constraint chk_notificaciones_estado
            check (estado in ('PENDIENTE', 'ENVIADA',
'LEIDA', 'FALLIDA'))
    );

create index if not exists idx_vehiculos_usuario_id
    on vehiculos(usuario_id);

create index if not exists idx_horarios_taller_taller_id
    on horarios_taller(taller_id);

create index if not exists
idx_taller_especialidades_taller_id
    on taller_especialidades(taller_id);

create index if not exists
idx_taller_especialidades_especialidad_id
    on taller_especialidades(especialidad_id);

create index if not exists idx_tecnicos_taller_id
    on tecnicos(taller_id);

create index if not exists
idx_tecnicos_estado_disponibilidad
    on tecnicos(estado_disponibilidad);

create index if not exists
idx_tecnico_especialidades_tecnico_id
    on tecnico_especialidades(tecnico_id);

create index if not exists
idx_tecnico_especialidades_especialidad_id
    on tecnico_especialidades(especialidad_id);

create index if not exists
idx_incidentes_cliente_usuario_id
    on incidentes(cliente_usuario_id);

create index if not exists idx_incidentes_vehiculo_id
    on incidentes(vehiculo_id);

create index if not exists
idx_incidentes_tipo_incidente_id
    on incidentes(tipo_incidente_id);

create index if not exists
idx_incidentes_estado_incidente_id

```

```

    on incidentes(estado_incidente_id);

create index if not exists idx_incidentes_fecha_solicitud
    on incidentes(fecha_solicitud);

create index if not exists idx_evidencias_incidente_id
    on evidencias_incidente(incidente_id);

create index if not exists idx_historial_incidente_id
    on historial_estados_incidente(incidente_id);

create index if not exists idx_historial_estado_id
    on historial_estados_incidente(estado_incidente_id);

create index if not exists idx_candidatos_incidente_id
    on candidatos_taller_incidente(incidente_id);

create index if not exists idx_candidatos_taller_id
    on candidatos_taller_incidente(taller_id);

create index if not exists idx_asignaciones_taller_id
    on asignaciones_servicio(taller_id);

create index if not exists idx_asignaciones_tecnico_id
    on asignaciones_servicio(tecnico_id);

create index if not exists
idx_asignaciones_especialidad_id
    on asignaciones_servicio(especialidad_id);

create index if not exists idx_asignaciones_estado
    on asignaciones_servicio(estado_asignacion);

create index if not exists idx_pagos_cliente_usuario_id
    on pagos(cliente_usuario_id);

create index if not exists idx_pagos_estado_pago
    on pagos(estado_pago);

create index if not exists
idx_calificaciones_cliente_usuario_id
    on calificaciones(cliente_usuario_id);

create index if not exists idx_notificaciones_usuario_id
    on notificaciones(usuario_id);

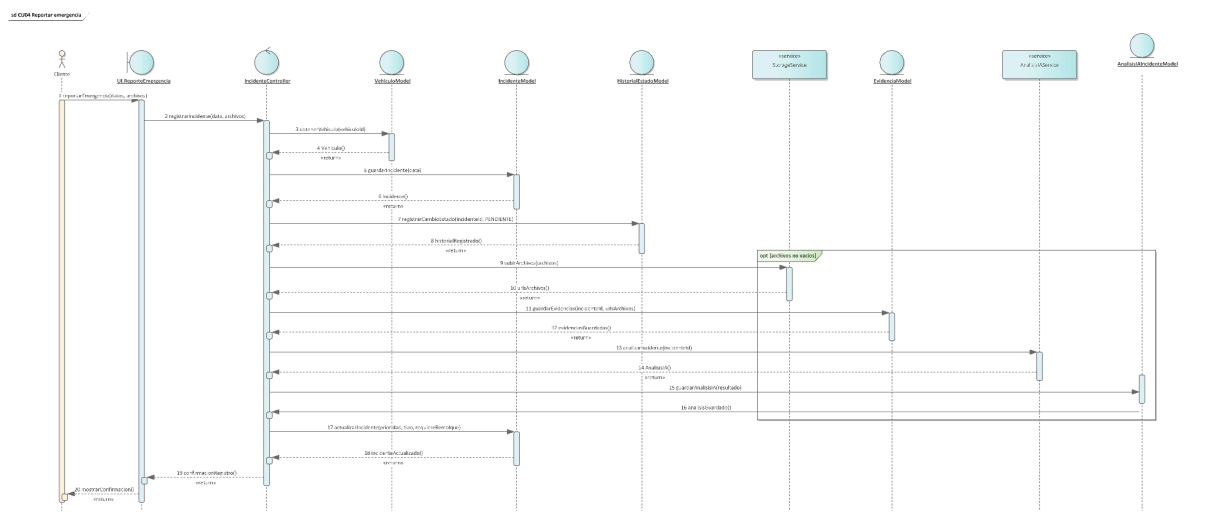
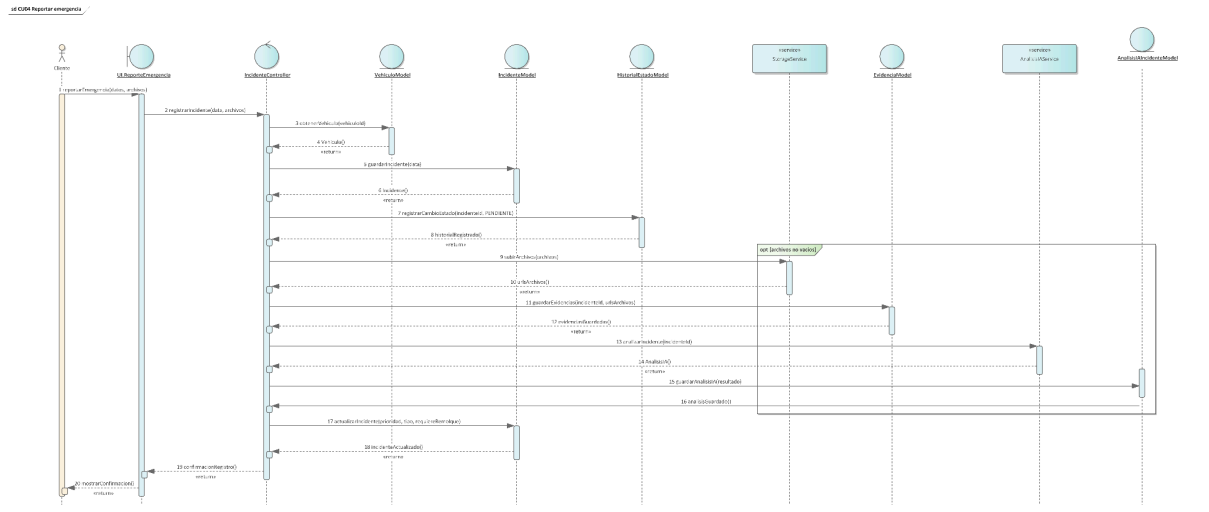
create index if not exists
idx_notificaciones_incidente_id
    on notificaciones(incidente_id);

create index if not exists idx_notificaciones_estado

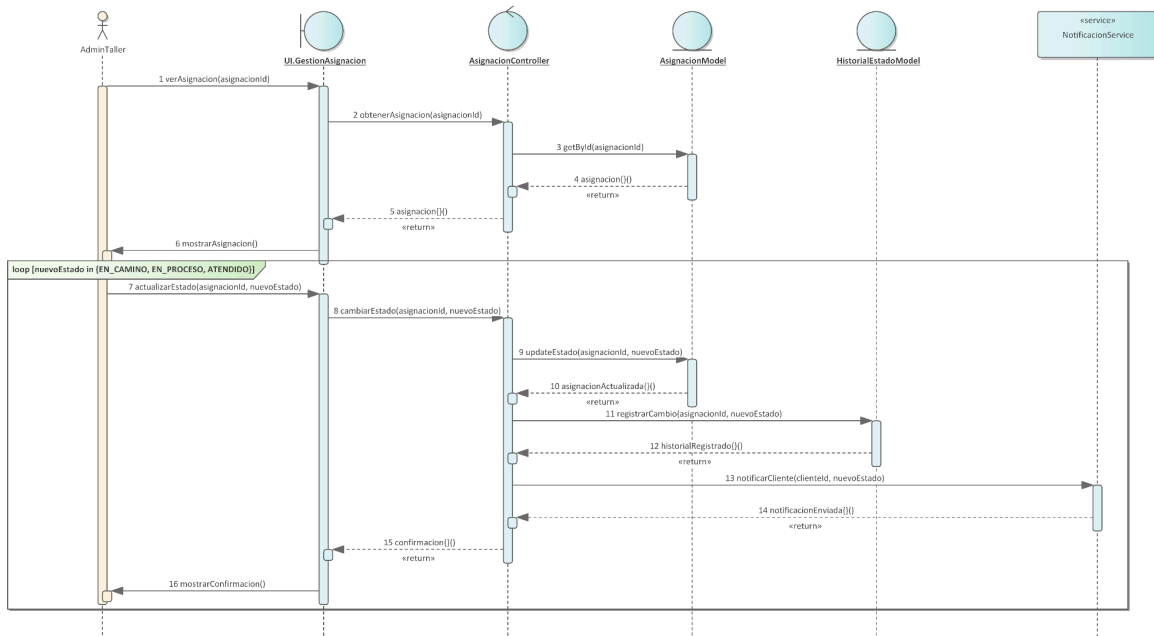
```

```
on notificaciones(estados);
```

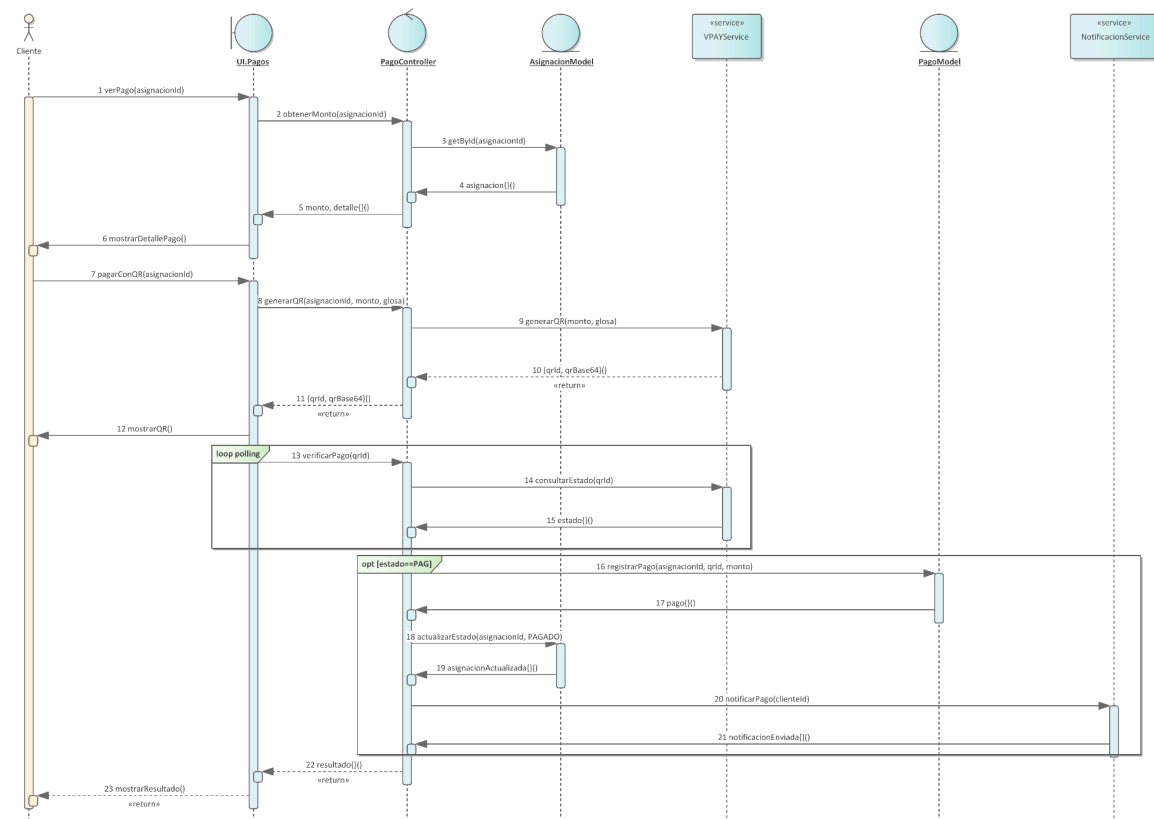
3.3.3 Diagramas de Secuencia



sd CU14 Gestionar atencion del servicio



sd CU19 Registrar pago



3.4 FT: IMPLEMENTACIÓN

3.4.1 Elección de plataforma de desarrollo de software

La elección tecnológica se definió considerando compatibilidad con el enunciado del proyecto, rapidez de desarrollo, disponibilidad de librerías y facilidad de despliegue.

Backend

Se selecciona FastAPI por su rendimiento, documentación automática, validación de datos mediante modelos y facilidad para construir APIs modernas.

Frontend web

Se selecciona Angular por su estructura modular, capacidad para aplicaciones administrativas complejas y facilidad de integración con APIs y componentes visuales.

Aplicación móvil

Se selecciona Flutter por su desarrollo multiplataforma, acceso a funcionalidades nativas y rapidez para prototipado e implementación.

Base de datos

Se selecciona PostgreSQL por su robustez, soporte relacional, integridad y facilidad de modelado para trazabilidad.

Infraestructura

Se selecciona AWS para almacenar evidencias, desplegar backend y alojar base de datos de forma escalable.

3.4.2 Implementación de la arquitectura del sistema

Diagrama de Componentes

3.4.3 Endpoints sugeridos

Algunos endpoints base recomendados son:

- POST /auth/login
- POST /clients
- POST /vehicles
- POST /incidents
- GET /incidents/{id}
- POST /incidents/{id}/evidences
- GET /workshops/available-incidents
- POST /workshops/incidents/{id}/accept

- POST /assignments
- PATCH /incidents/{id}/status
- POST /payments
- POST /ratings

3.4.4 Organización inicial de repositorios

Con el fin de mantener una estructura clara de desarrollo, despliegue y defensa académica, el proyecto se organizará en tres repositorios principales:

Repositorio 1: Backend FastAPI

Propósito: centralizar la lógica de negocio, autenticación, gestión de incidentes, talleres, técnicos, pagos, almacenamiento de evidencias y servicios inteligentes de clasificación y asignación.

Responsabilidades principales:

- exponer la API REST del sistema;
- gestionar seguridad y autenticación;
- administrar persistencia en **PostgreSQL**;
- **integrar almacenamiento y servicios externos**;
- concentrar reglas de negocio y trazabilidad.

Repositorio 2: Frontend Web Angular

Propósito: implementar el panel de operación y administración para talleres.

Responsabilidades principales:

- autenticación de usuarios del taller;
- gestión de perfil del taller;
- administración de técnicos y **disponibilidad**;
- **visualización y atención** de solicitudes;
- consulta de métricas, historial y estados.

Repositorio 3: Aplicación Móvil Flutter

Propósito: implementar la experiencia del cliente final en dispositivos móviles.

Responsabilidades principales:

- autenticación y perfil del cliente;
- registro de vehículos;
- registro de emergencias con evidencias y ubicación;
- seguimiento del servicio;
- pago, historial y calificación.

3.4.5 Estructura base sugerida por repositorio

Backend FastAPI

Estructura sugerida:

- app/
- app/api/
- **app/core/**
- app/models/
- app/schemas/
- app/services/
- app/repositories/
- app/utils/
- tests/

Frontend Angular

Estructura sugerida:

- src/app/core/
- src/app/shared/
- src/app/features/auth/
- src/app/features/workshops/
- src/app/features/**technicians/**
- src/app/features/incidents/
- src/app/features/payments/
- src/app/features/reports/

Aplicación Flutter

Estructura sugerida:

- lib/config/
- lib/core/
- lib/shared/
- lib/features/auth/
- lib/features/vehicles/
- lib/features/incidents/
- lib/features/tracking/
- lib/features/payments/
- lib/features/history/

Además, se recomienda una convención de ramas simple y defendible para la materia:

- main: versión estable;
- develop: integración del equipo;
- feature/nombre-modulo: desarrollo de funcionalidades específicas.

3.5 Pruebas

3.5.1 Estrategia de pruebas

Las pruebas del sistema deben abarcar validación funcional, integración entre módulos y consistencia de reglas de negocio. Para el primer parcial se recomienda **documentar casos de prueba clave orientados al flujo principal**.

3.5.2 Casos de prueba sugeridos

Caso de prueba 1: Registro de emergencia exitoso

Objetivo: **verificar que** un cliente autenticado pueda registrar una **emergencia con vehículo**, ubicación y evidencia.

Precondición: cliente autenticado con vehículo registrado.

Resultado esperado: el sistema crea el incidente, almacena la evidencia y devuelve el estado Registrada.

Caso de prueba 2: Taller acepta solicitud disponible

Objetivo: validar que un **taller pueda aceptar** una solicitud activa y que esta deje de **aparecer como disponible** para otros talleres.

Precondición: incidente **clasificado y publicado a talleres candidatos**.

Resultado esperado: el incidente queda asociado al taller aceptante.

Caso de **prueba 3:** Asignación de técnico disponible

Objetivo: verificar la asignación de un técnico al incidente aceptado.

Precondición: taller con al menos un técnico disponible.

Resultado esperado: el **técnico queda** asociado al incidente y su disponibilidad cambia.

Caso de prueba 4: Actualización de estado del servicio

Objetivo: comprobar que el administrador del taller pueda cambiar el estado del incidente y que el cambio se registre en el historial.

Precondición: incidente asignado.

Resultado esperado: el nuevo estado queda almacenado y notificado al cliente.

Caso de prueba 5: Registro de pago y cierre

Objetivo: validar **que el sistema permita** registrar el pago y cerrar la atención.

Precondición: incidente finalizado operativamente.

Resultado esperado: pago registrado, estado cerrado y habilitación de calificación.

3.5.3 Criterios de aceptación del MVP

Para considerar que el producto mínimo viable del primer parcial se encuentra correctamente definido y listo para pasar a implementación, deben cumplirse los siguientes criterios:

1. Debe existir autenticación funcional para cliente y taller.
2. El cliente debe poder registrar al menos un vehículo.
3. El cliente debe poder registrar una emergencia con ubicación y al menos un tipo de evidencia.
4. El sistema debe almacenar el incidente y dejarlo disponible para análisis y atención.
5. El taller debe poder visualizar solicitudes compatibles con su contexto de operación.
6. El taller debe poder aceptar una solicitud y asignar un técnico disponible.
7. El servicio debe poder avanzar entre estados controlados por reglas definidas.
8. El cliente debe poder consultar el estado actualizado de su solicitud.
9. Debe existir un registro básico del pago y del cierre del servicio.
10. Debe existir persistencia trazable de incidentes, estados, asignaciones y pagos.

Estos criterios no implican que todos los módulos complementarios estén completamente desarrollados, sino que el flujo principal del sistema debe funcionar de extremo a extremo de forma demostrable.

CAPÍTULO 4: MANUAL DE USUARIO

4.1 Introducción

El presente manual tiene como finalidad orientar a los usuarios en el uso básico de la plataforma inteligente de atención de emergencias vehiculares. Debido a que el

sistema cuenta con dos interfaces principales —una móvil para clientes y una web para talleres—, el manual se organiza de acuerdo con el rol de usuario.

4.2 Acceso a la aplicación

4.2.1 Versión móvil – clientes

El cliente debe descargar la aplicación móvil, crear una cuenta o iniciar sesión con sus credenciales. Una vez dentro, podrá registrar vehículos, crear una solicitud de emergencia, revisar el estado de sus servicios y acceder a su historial.

4.2.2 Versión web – talleres

El taller debe ingresar mediante navegador web utilizando sus credenciales. Desde el panel podrá revisar solicitudes, gestionar técnicos, actualizar el estado del servicio y consultar el historial de atenciones.

4.3 Uso de funcionalidades

4.3.1 Registro de vehículo

1. Ingresar al módulo “Mis vehículos”.
2. Seleccionar la opción “Agregar vehículo”.
3. Completar placa, marca, modelo, año y color.
4. Guardar la información.

4.3.2 Registro de emergencia

1. Seleccionar el vehículo afectado.
2. Presionar “Nueva emergencia”.
3. Permitir acceso a ubicación.
4. Adjuntar foto, audio o texto explicativo.
5. Confirmar el envío.

4.3.3 Consulta del estado del servicio

1. Ingresar al historial o a la solicitud activa.
2. Visualizar el estado actual.
3. Revisar el taller asignado y las observaciones.

4. Esperar nuevas actualizaciones notificadas por el sistema.

4.3.4 Aceptación de solicitud por el taller

1. Ingresar al panel de solicitudes disponibles.
2. Revisar el resumen del incidente.
3. Evaluar disponibilidad de recursos.
4. Aceptar o rechazar la solicitud.

4.3.5 Asignación del técnico

1. Ingresar al detalle de la solicitud aceptada.
2. Seleccionar uno de los técnicos disponibles.
3. Confirmar la asignación.
4. Verificar que el estado cambie correctamente.

4.3.6 Actualización de estado

1. Abrir la atención activa.
2. Seleccionar el nuevo estado.
3. Registrar observaciones si corresponde.
4. Guardar el cambio para notificar al cliente.

4.3.7 Registro de pago y calificación

1. Una vez finalizado el servicio, el sistema mostrará el costo final.
2. El cliente seleccionará el método de pago.
3. Se registrará la transacción.
4. El cliente podrá dejar una calificación y comentario.

4.4 Notificaciones

El sistema enviará notificaciones push o internas cuando:

- la solicitud sea registrada;
- un taller acepte la atención;
- un técnico sea asignado;
- cambie el estado del servicio;
- el pago quede confirmado;
- se habilite la calificación final.

4.5 Recomendaciones de uso

- Mantener actualizada la información del vehículo.
- Permitir acceso a ubicación para mejorar la asignación.
- Adjuntar fotografías claras del incidente.
- Describir la falla con el mayor detalle posible.
- Verificar la conexión a internet durante el registro de emergencias.

CONCLUSIONES

La propuesta de desarrollar una plataforma inteligente de atención de emergencias vehiculares constituye una respuesta pertinente ante la falta de organización, trazabilidad y eficiencia en los servicios de auxilio mecánico. El análisis realizado permite concluir que el problema no radica únicamente en la existencia de fallas vehiculares, sino en la forma fragmentada en que actualmente se gestionan las solicitudes, la asignación de proveedores y el seguimiento del servicio.

El proyecto integra componentes clave de un sistema de información moderno: aplicaciones web y móviles, servicios orientados a APIs, almacenamiento estructurado, geolocalización, notificaciones y módulos de inteligencia artificial aplicados al flujo principal. Esto lo convierte en una propuesta coherente con las exigencias académicas de la materia y con posibilidades reales de aplicación práctica.

Asimismo, el enfoque metodológico basado en PUDS y UML permite organizar el proyecto desde la captura de requisitos hasta el diseño e implementación, aportando claridad, trazabilidad y justificación técnica en cada decisión tomada.

RECOMENDACIONES

- Completar en la siguiente fase los diagramas UML formales: casos de uso, clases, secuencia, actividad, componentes y despliegue.
- Refinar el modelo de datos físico con tipos, claves foráneas e índices.
- Definir reglas de negocio detalladas para la prioridad del incidente y la selección de talleres candidatos.

- Limitar el alcance del MVP a un conjunto inicial de incidentes frecuentes, como batería, llanta, choque leve y remolque, para asegurar una implementación funcional y defendible.
- Implementar primero el flujo de extremo a extremo antes de ampliar funcionalidades complementarias.
- Preparar una defensa centrada en el valor del flujo completo: registro de emergencia, análisis, asignación, atención, pago y cierre.

BIBLIOGRAFÍA

Booch, G., Rumbaugh, J., & Jacobson, I. The Unified Modeling Language User Guide.

Pressman, R. Ingeniería de Software: un enfoque práctico.

Sommerville, I. Ingeniería del Software.

Documentación oficial de FastAPI.

Documentación oficial de Angular.

Documentación oficial de Flutter.

Documentación oficial de PostgreSQL.

Documentación oficial de Amazon Web Services (AWS).

Fuentes especializadas sobre geolocalización, notificaciones push y arquitecturas basadas en servicios.

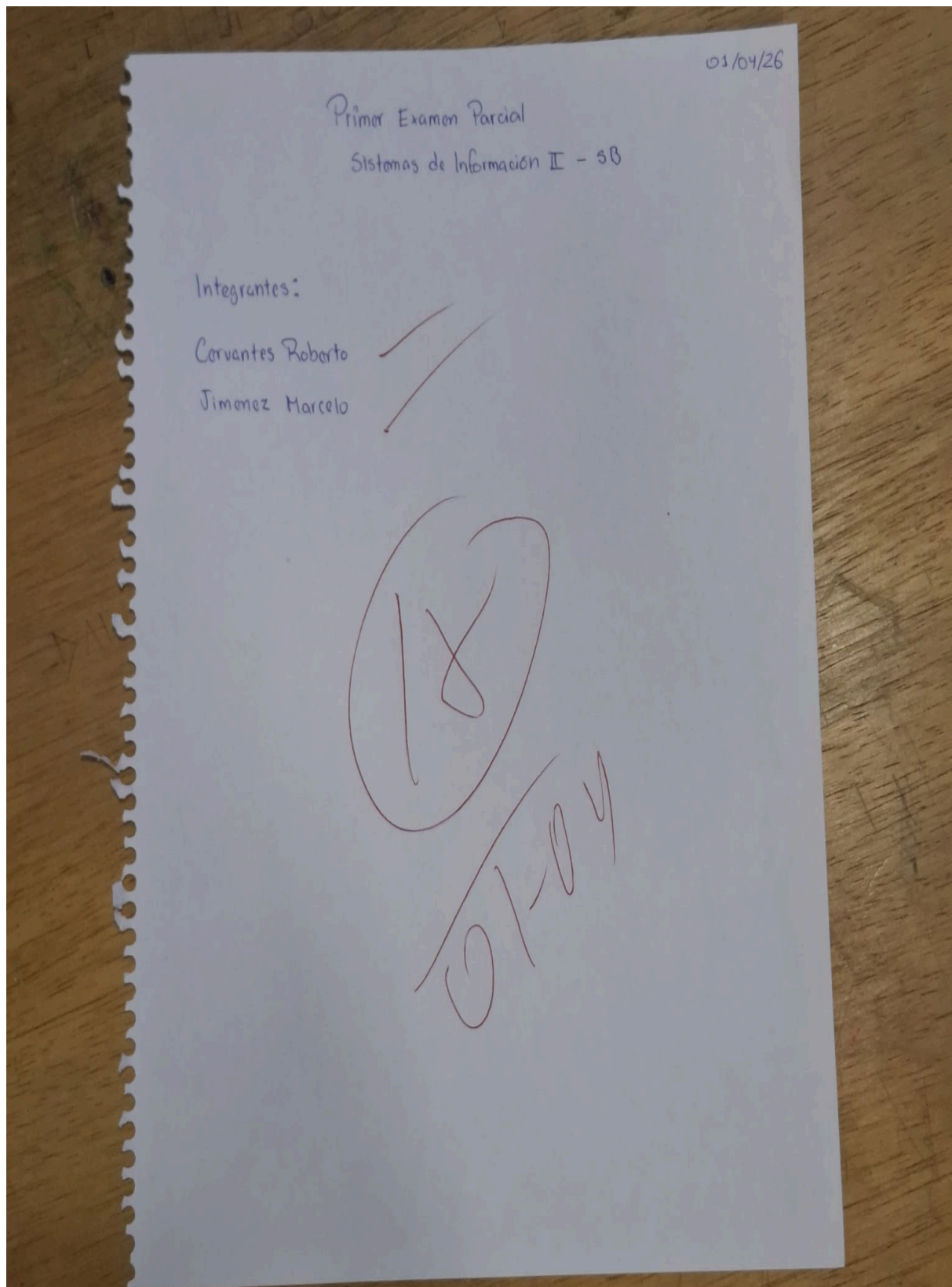
URL Y QR

[Agregar enlace del repositorio]

[Agregar enlace del despliegue]

[Agregar código QR de descarga o acceso]

ANEXOS



- Mockups de la aplicación móvil.
- Mockups del panel web de talleres.

- Capturas de pruebas funcionales.